

ROADMAP — Secret-Sauce

All implementation tasks, grouped by phase. Status is kept in sync with the code (see the "Docs-code sync" rule in [CLAUDE.md](#)).

Legend: [] not started · [~] in progress · [x] done · [→] moved elsewhere — to [Backlog](#), or to a later phase that now owns it (the item names which)

Product direction (the north star)

Secret-Sauce is a **recipe vault first**, and that stays the core. The end state adds one more layer of identity on top of the chef layer that Phases 18–23 built:

1. **Recipes** — structured, forkable, versioned. (*Phases 2–20: done*)
2. **Chefs** — a presentation of **profiles**, scored and tiered by engagement with their public recipes. (*Phases 18, 22, 23: done*)
3. **Restaurants** — a new entity. A restaurant lists **member chefs** (association is optional — most users never join one) and **signature dishes**, which are pointers into the existing **recipes** table, not a second recipe system. (*Phase 25: designed, not started*)

The build order to date is deliberately compatible with that end state: "chef" was built as a presentation of **profiles** rather than a second principal table, and a restaurant will be built as an entity *managed by* profiles rather than a login of its own — so auth, RLS, and the engagement model all stay single-principal. The main gaps between here and Phase 25 are listed in that phase's prerequisites (most notably: there is still no public chef page, and Phase 25's restaurant page needs the same shape).

Phase 0 — Documentation foundation

- ☒ Create **CLAUDE.md**
- ☒ Create **docs/ROADMAP.md**
- ☒ Create **docs/EXECUTION-PLAN.md**
- ☒ Create **docs/SDS.md**
- ☒ Create **docs/BUG-TRACKER.md**

Phase 1 — Monorepo scaffold

- ☒ **melos.yaml** + root **pubspec.yaml** (Dart workspace)
- ☒ **packages/core** package skeleton + **pubspec.yaml**
- ☒ **packages/design_system** package skeleton + **pubspec.yaml**
- ☒ **apps/app** application skeleton + **pubspec.yaml**
- ☒ **analysis_options.yaml** (shared lints)
- ☒ **melos bootstrap** resolves cleanly
- ☒ CI workflow (analyze + test) — **.github/workflows/ci.yml**

Phase 2 — Supabase schema + RLS

- ☒ `profiles` table + trigger
- ☒ Enums: `difficulty`, `recipe_visibility`, `share_permission`, `suggestion_status`
- ☒ `recipes` table (+ fork lineage columns)
- ☒ `recipe_versions` table (git-like snapshots)
- ☒ `ingredient_groups` + `ingredients`
- ☒ `step_groups` + `steps`
- ☒ `tags` + `recipe_tags`
- ☒ `recipe_shares`
- ☒ `recipe_likes`, `recipe_saves`, `recipe_views`
- ☒ `recipe_ratings` (half-star 0.5–5.0) + aggregate trigger on `recipes.rating_*`
- ☒ `recipe_suggestions` (reserved stub for future PR flow)
- ☒ RLS policies (public/private/shared)
- ☒ Storage buckets (recipe images, avatars)
- ☒ Apply migrations to a Supabase project — the hosted project carries the current `0001_init.sql` in full (re-applied during the Phase 18 rollout)

Phase 3 — core package

- ☒ Models: `Profile`, `Recipe`, `IngredientGroup`, `Ingredient`, `StepGroup`, `RecipeStep`, `RecipeVersion`, `RecipeTag`, enums
- ☒ `SupabaseService` (client bootstrap)
- ☒ `AuthService` + `AuthRepository`
- ☒ `RecipeRepository` (CRUD, fork, versioning) contract + Supabase impl
- ☒ `DiscoverRepository` (popular/trending/recent/search) contract + impl
- ☒ `StorageService` (image upload)
- ☒ `packages/core/test/` exists — model/enum JSON decoding is covered (pure functions; no client needed)
- ☒ Unit tests for **repositories** — done by **OPT-T2**, and the blocker turned out not to exist: `SupabaseClient` takes an `httpClient`, so `packages/core/test/support/fake_supabase.dart` records the request and replies from a queue under a **real** client. 14 tests at first; no mock. **33 now** across four repositories — **food** landed with Phase 29a, and **chef** with the Phase 18 coverage sweep (2026-08-25), which was the last one still unwritten

Phase 4 — design_system

- ☒ `AppTheme` (light/dark, tokens)
- ☒ `RecipeCard` (image, name, description, time, difficulty)
- ☒ `DifficultyBadge`
- ☒ Adaptive helpers (`Breakpoints`, `AdaptiveLayout`)
- ☒ Common widgets (buttons, loading, empty states)

Phase 5 — app shell + auth

- ☒ `main.dart` bootstrap (Supabase init, ProviderScope)

- ☒ `go_router` config + responsive `AppShell` (top nav bar / bottom nav)
- ☒ Auth controller (Riverpod)
- ☒ Sign in / Sign up screens
- ☒ Auth guard / redirect

Phase 6 — Home + Discover

- ☒ ~~Home / landing (intro, feature highlights, sign-in/up)~~ — **retired 2026-08-20**. No navigation chrome ever linked it (the web brand mark goes to Discover), so a cold start was the only way in. `/` is now a redirect-only route onto `/discover`.
- ☒ Discover screen (Popular / Trending / Recent tabs) — the front door on both platforms
- ☒ Search bar + results
- ☒ Public browsing: Discover + public recipe detail accessible without sign-in

Phase 7 — My Recipes + sharing

- ☒ My Recipes screen (My / Shared-with-me tabs)
- ☒ Recipe grid using `RecipeCard`
- ☒ Share dialog (add users, set permission)

Phase 8 — Recipe Detail

- ☒ Recipe detail screen (structured ingredients + steps)
- ☒ Servings scaler
- ☒ Like / save actions
- ☒ Fork button entry point
- ☒ Version history view

Phase 9 — Recipe Editor

- ☒ Create/edit form (metadata)
- ☒ Ingredient-group + ingredient editor
- ☒ Step-group + step editor
- ☒ Per-step time / temperature / tip and per-ingredient note / optional inputs (B035)
- ☒ Cover image upload
- ☐ Per-step image upload — the column is carried through a save untouched, but there is no picker for it yet (B035)
- ☒ Save → creates new `recipe_version`

Phase 10 — Fork + version history

- ☒ Fork action (copy + record lineage)
- ☒ "Forked from ..." attribution on detail
- ☒ Version diff/history browsing

Phase 11 — Search + ranking

- ☒ Full-text search (title/ingredient/tag)

- ☒ Trending ranking (recency-weighted likes/views)
- ☒ Popular ranking — **now rating-based** (Bayesian weighted average; saves/likes only break ties)
- ☒ **view_count** rollup trigger — counts distinct signed-in viewers, ignores anonymous and repeat views (advisory-locked against concurrent first-views), so trending is not inflatable (B012)
- ☒ **SQL regression harness — built, and it is shape (2)**. This was the largest untested surface in the project: the B012 view trigger, every RLS policy, the rank RPCs and the whole chef score/tier/leaderboard layer were verified only by manual local-stack runs recorded in [docs/BUG-TRACKER.md](#), and the failure mode for the trigger/RLS class is *silence*, not an error (B011, B012). The recommendation was "a suite **plus a CI job**, on the full Supabase stack because [profiles.id](#) references [auth.users](#)". That is what shipped, across four pieces built in different phases:

```
- ``.github/workflows/database.yml`` (OPT-T1) — starts the stack inside
the runner and
  walks three paths: fresh apply, full re-apply (the only check of
the idempotency every
  file claims), and the Gotcha 6 upgrade path that B024 shipped
through. No
  `SUPABASE_DB_URL` secret, by design (Gotcha 7)
- ``supabase/tests/rls_matrix.sql`` (BL-7, `melos run db:rls`) — 92
checks as a signed-in
  user across anon / owner / shared-with / stranger, rolled back. Every
other SQL step in CI
  runs as `postgres`, which bypasses policies; this is the only thing
that does not. Found
  B061 on its first complete run
- ``supabase/sim/3_sim_verify.sql`` (Phase 24) — 46 assertions over a
generated `tiny`
  population: counter invariants incl. B012's anonymous/repeat-view
exclusion, authorization,
  temporal, structural and shape
- ``supabase/tests/nutrition_{estimate,fixtures}.sql`` (Phase 29c/29d)
— the estimator's
  arithmetic on fixture trees, and the committed labels re-estimated
against the real
  registry as a drift gate

What is still not covered, so this item closing does not overstate
itself: Storage bucket
policies (needs the storage container and a real upload, not SQL); the
PostgREST edge (the
matrix talks to Postgres directly, so it proves the policy, not that
postgrest-dart sends what
the policy expects — `packages/core/test/` is the other half); and there
is no aggregate
```

```
`melos run test:sql` – the suites run as `db:rls` / `db:sim:verify` /  
`db:nutrition:estimate` /  
`db:nutrition:verify`, four scripts, all four wired into CI
```

Phase 12 — Polish, tests, verification

- ☒ Widget tests (`RecipeCard`, root / → `/discover` redirect) — passing. The Home landing test was replaced by the redirect test when that screen was retired.
- ☒ Repository unit tests — landed with **OPT-T2** (recording `http.BaseClient`, not a mock); see Phase 3
- ☒ `melos run analyze` clean (core, design_system, app)
- ☒ Manual verification on web (`web-server` @ :8080) — sign-up works
- ☐ Manual verification on mobile/emulator (*pending*)
- ☒ Empty/error/loading states pass review

Phase 13 — Toolchain / reproducible setup

- ☒ Document Windows Flutter SDK install from scratch (no `winget` package) — `README.md`
- ☒ Pin toolchain to Flutter 3.44.8 / Dart 3.12.2 + melos 6.3.3, with rationale
- ☒ Pin CI to the same versions (`.github/workflows/ci.yml` no longer tracks `channel: stable`)
- ☒ Add a setup troubleshooting table to `README.md`
- ☒ Log toolchain bugs B005–B010 in `docs/BUG-TRACKER.md`
- ☒ Close the `.gitignore` gap that left extension-less `env.local` credential files tracked (B010)
- ☒ End-to-end runtime check: web server + Supabase Auth + PostgREST seeded data
- ☒ Commit `pubspec.lock` files so resolution is reproducible (B009 — *done in OPT-T4*)
- ☐ Migrate to `freezed` 3.x so the project can track current Flutter stable (*unblocks B005*)

Phase 14 — Ratings

- ☒ `recipe_ratings` table + half-star check constraint (0.5 ... 5.0)
- ☒ `recompute_recipe_rating()` trigger keeping `recipes.rating_sum` / `rating_count` / `rating_avg` exact
- ☒ RLS: rate only your own row, only recipes you can read, never your own recipe
- ☒ `Recipe.ratingAvg` / `ratingCount` on the model
- ☒ `RecipeRepository.myRating` / `setRating` / `clearRating`
- ☒ `StarRating`, `RatingPill`, `StarRatingInput` widgets in `design_system`
- ☒ Rating shown on `RecipeCard`; rate/remove UI on recipe detail
- ☒ Popular ranking switched to the Bayesian rating score
- ☒ Seed data: 8 "taster" accounts supply believable ratings on the curated recipes
- ☒ Widget tests for star rendering, half-star snapping, and range clamping
- ☒ Fix B016: `RecipeCard` metadata row overflowed once the rating pill was added — row is now fully flexible; regression tests cover 276/320px at 1.0x and 2.0x text scale
- ☒ Fix B017: `StarRatingInput` kept a stale preview when a scroll cancelled the gesture
- ☒ Fix B018: seed accounts no longer carry literal passwords — randomized, since `seed.sql` is documented as safe to run against the hosted project

- ☐ Rotate/delete the 9 seed accounts on any **already-seeded** database (*B018 — the seed's on conflict do nothing leaves existing password hashes in place*). **Now more pressing:** the hosted project is live and those 9 rows predate the fix, so if they were created from the pre-B018 seed they still carry the committed literal passwords. The 7 Phase 18 chef accounts are **not** affected — they were randomized from their first insert.
- ☒ Fix B011: counter/aggregate triggers made **security definer** (RLS was silently dropping like/save updates on recipes the acting user doesn't own)
- ☒ Fix B013: explicit table grants for **anon** / **authenticated** (current Supabase images no longer grant DML on new tables by default)
- ☒ Verified on a local Supabase stack: schema + seed apply clean, aggregates/ranking correct, RLS rejects self-rating, anon rating, and bad values — see [docs/BUG-TRACKER.md](#)
- ☒ Re-apply the updated **0001_init.sql** to the **hosted** project (idempotent) — done during the Phase 18 rollout, so the hosted project now carries the B012 view-count trigger and the tightened **views_insert** policy as well as the chef objects

Phase 15 — Visibility polish

- ☒ Public/Private badge on **RecipeCard** (**showVisibility**), enabled on the My Recipes tab
- ☒ Confirmed end-to-end: editor toggle → **recipes.visibility** → RLS **can_read_recipe()** → Discover only lists **public**; private recipes reach only the owner + **recipe_shares** users

Phase 16 — Agent-context accuracy

- ☒ Audit **CLAUDE.md** against the code; correct the repository tree (**src/** layer), the import rule (**always_use_package_imports**), the enum list, and the Riverpod codegen claim (B019–B021)
- ☒ Add a "Gotchas & invariants" section to **CLAUDE.md** citing B005–B018, plus a routes/feature map and a real-vs-cosmetic enforcement table
- ☒ Fix the pre-**src/** paths in **docs/EXECUTION-PLAN.md** Phases 3–4 (B020)
- ☒ Drop the unused **riverpod_annotation** / **riverpod_generator** deps from **core** + **app**, and **build_runner** from **app** (no annotated sources there) (B021)
- ☒ Document the **packages/core** test-coverage hole in **CLAUDE.md** (repositories, **snapRating**, and JSON decoding are untested; **melos run test** says nothing about **core**)

Phase 17 — Fix B022: nested content order reversed

- ☒ Add **ascending: true** to the four nested-content **.order()** calls in **SupabaseRecipeRepository._fetchIngredientGroups** / **_fetchStepGroups** (postgrestdart defaults to descending — B022)
- ☒ Audit stored order of any recipe **edited through the app**: each save round-trip flipped the persisted **sort_order/step_order** (odd number of edits ⇒ reversed in storage); repair manually — parity is unknowable, so no auto-fix.
Result: nothing to repair. All 6 recipes on the hosted project carry exactly one **recipe_versions** row (**version_number = 1, 'Seeded recipe'**), so none was ever saved through the editor. Caveat recorded in **BUG-TRACKER.md**: the one real account's *private* recipes are not anon-readable and could not be inspected.

- ☒ Verify on the local Supabase stack (core has no tests — a green `melos run test` proves nothing here): recipe with 5+ steps, 2+ groups displays 1→N; edit + save twice; re-check
- ☒ Close B022 in `docs/BUG-TRACKER.md`

Phase 18 — Chefs, tiers & leaderboard

Design: [SDS.md §10](#) ·

Execution: [EXECUTION-PLAN.md Phase 18](#)

Schema (`0001_init.sql`, idempotent)

- ☒ `chef_tier` enum (guarded) — five tiers `home_cook ... master_chef`
- ☒ `profiles.chef_score / chef_tier / public_recipe_count` via `add column if not exists`
- ☒ `chef_score()` + `chef_tier_for()` immutable functions (single source of truth)
- ☒ `recompute_chef_stats(uuid)` — invoker-rights, EXECUTE revoked from API roles
- ☒ `on_recipe_stats_change` trigger on `recipes` — **security definer set search_path = public** (writes other users' `profiles` rows; B011 class), handles insert/delete/update incl. `visibility` and `owner_id` changes
- ☒ Idempotent backfill recomputing all profiles on every apply
- ☒ `chefs_leaderboard(p_limit, p_offset)` RPC — `dense_rank`, deterministic tie-breaks, excludes `public_recipe_count = 0`, explicit `visibility = 'public'` filter
- ☒ `drop.sql`: new functions/trigger/type + the old `seed_recipe` signature
- ☒ Grants check: new function EXECUTE for `anon/authenticated` on `chefs_leaderboard` only (B013 class — the blanket grant block runs before these objects exist on first apply)

core

- ☒ `ChefTier` Dart enum (5 values, `unknownEnumValue` fallback) — updates the "four enums" note in `CLAUDE.md`
- ☒ `Profile` + `chefScore / chefTier / publicRecipeCount; Recipe` + embedded `Profile? owner`; codegen re-run
- ☒ Owner embedding added to recipe list/detail/RPC queries — as the shared `kRecipeSelect` constant in `recipe_queries.dart`. **The plain `owner:profiles(...)` form does not work:** `recipes` and `profiles` are related five ways, so PostgREST answers `PGRST201` unless the FK is named (`owner:profiles!recipes_owner_id_fkey(...)`)
- ☒ `ChefStanding` model + `ChefRepository` (abstract + Supabase impl, provider wiring)

design_system

- ☒ `TierChip` (theme-aware tier colors, light + dark)
- ☒ `ChefBadge` (avatar + name + tier **under** the name; `compact` variant)
- ☒ Barrel exports for both (Gotcha 13)
- ☒ `RecipeCard`: chef badge as **cover overlay, bottom-left** — never a new column row (B001/B002/B016); envelope tests at 276 px / 2.0× text scale (v2 *moved it to bottom-right* — Phase 20)

app

- ☒ `/chefs` route in the `ShellRoute` + `AppShell` destination (signed-out safe — no redirect change)
- ☒ `features/chefs/`: leaderboard screen + providers (loading/empty/error states)
- ☒ Recipe detail: full `ChefBadge` under the title

Seed + verification

- ☒ Seed chefs d1–d7 (fixed UUIDs, randomized passwords — B018): one per tier, exact-threshold boundary, zero-engagement, private-only, tied-score pair
- ☒ `seed_recipe()` + `p_visibility` param (old signature → `drop.sql`)
- ☒ Local-stack verification run recorded in `BUG-TRACKER.md`: idempotent double-apply, tier boundaries, trigger fires for non-owner engagement (definer check), leaderboard exclusions/ties, anon RPC call
- ☒ Widget tests: `TierChip`, `ChefBadge`, card overlay envelope, leaderboard screen states; `melos run analyze + test --no-select clean` (35 tests)
- ☒ Fix B023: two `RenderFlex` overflows in the leaderboard row, found by the new 320 px / 2.0× envelope test before merge
- ☒ Docs sync: SDS §3/§4/§7/§8/§10 fold-in, `CLAUDE.md` feature map + enum count + server-owned columns list + FK-hint gotcha (`README.md` unchanged — no command changes)
- ☒ Coverage audit + gap fill: added `packages/core/test/` (**first tests in core** — model decoding is pure, so the `SupabaseClient`-mocking blocker never applied to it) covering the `ChefTier` wire format, the `unknownEnumValue` fallback, `ChefStanding` / `Profile` / `Recipe.owner` decoding, `numeric`-as-int-or-double, and a `kRecipeSelect` FK-hint guard; plus `chefs_routing_test.dart` pinning `/chefs` as signed-out safe while the guarded routes stay guarded. 35 → 65 tests
- ☒ Re-apply `0001_init.sql` to the **hosted** project — applied; backfill scored all 10 existing profiles, Kitchen lands `head_chef` at 10197
- ☒ Fix B024: `seed.sql` failed on any **already-seeded** database — the `seed_recipe` signature change left two overloads and every call matched both. Drops now live in `seed.sql` itself, not only `drop.sql`
- ☒ Re-run the fixed `seed.sql` on the **hosted** project — done. Verified over the anon API: 7 leaderboard rows, all five tiers, Chen Wei / Greta Lindqvist share rank 4, Kitchen at 10197 (8 above the local 10189 — the hosted DB carries one extra like and save from the B011 verification run, which is exactly 3+5)

Test coverage — remaining gaps (deferred)

Widget + model coverage landed with the feature (65 tests). These are what is still uncovered; none blocks the feature, all are real regression surface.

- [~] **SQL regression harness for the chef objects** — mostly covered now, via Phase 11's entry (closed) rather than as its own file. What **is** re-verified on every CI run: `chef_score()` over each profile's public recipes (`3_sim_verify.sql` **A5**, which is also the Gotcha 19 guard — a restated 3/5/0.2 fails it), `chef_tier_for()` agreeing with the stored tier (**A6**), the OPT-P5 totals not drifting (**A6b**), the inclusive-at-100 boundary (**F3** — Dara pinned at exactly 100), `on_recipe_stats_change` and public-only filtering implicitly (A5 is computed over `visibility = 'public'` and fails if the trigger stops

firing), and `recompute_chef_stats` / `recompute_all_chef_stats` being **unreachable** as a signed-in user (`rls_matrix.sql` **B17/B18**, 42501).

The three that were still unasserted are now covered — Phase 30 added them as `rls_matrix.sql` section **F**, since it was adding `chef_standing` checks to the same file: `dense_rank` ties (**F6**), the `public_recipe_count = 0` exclusion from both directions (**F4/F5**), and `anon` specifically being able to call `chefs_leaderboard` (**F1**) — which every prior run had proved as `postgres`, a role that bypasses grants and therefore proved nothing. Marked `[~]` rather than `[x]` only because `recompute_chef_stats`'s `is distinct from` guard still has no direct assertion.

The four Dart gaps are closed (2026-08-25). Each was blocked on something that has since been built — OPT-T2's recording HTTP client, the `RecipeRepository` fake that grew inside `recipe_detail_test.dart`, and Phase 30 giving `ChefBadge.onTap` a destination — so none of them needed new infrastructure, only the observation that the blocker had gone.

- ☒ `ChefRepository.leaderboard` unit test — `packages/core/test/chef_repository_test.dart`, **7 tests** covering all four methods, not just the board. What it pins: the RPC name and params for each, `kRecipeSelect`'s FK hint on `topRecipes` (Gotcha 17), `chef_score` arriving as an int or a double (Gotcha 12), `standing` decoding **zero rows to null** rather than the `PGRST116` a `.single()` would raise — the difference between "not ranked yet" and an error page — and every method running with no session (Gotcha 9)
- ☒ Recipe detail renders the owner badge — **both layouts**, since the badge has two call sites and Gotcha 26 says the second does not inherit the first's proof. `recipe_detail_test.dart` (compact) and `recipe_detail_v2_test.dart` (expanded) each assert the name and tier render, that no embed draws no badge, and that a tap lands on `/chef/:id` **with the owner's id** — the recipe's own id is on the same object, so a wrong one still navigates somewhere
- ☒ My Recipes passes `showChef: false` — `my_recipes_header_test.dart`, and the Shared-with-me tab asserted the other way (badge **on**, visibility pill off), because the two tabs disagree deliberately. Proven non-vacuous by flipping the flag: the My tab test goes red
- ☒ `ChefBadge` `onTap` / `onSurfaceImage` / `avatar-URL` branches. `onSurfaceImage` was already covered by B055's pair; the other two are now in `chef_badge_test.dart` — the callback fires and a badge **without** one grows no `InkWell`, and an `avatar_url` takes the `CachedNetworkImage` branch instead of the initials. The avatar path had never been rendered by any test or screenshot, because no seeded profile carries one (a standing BL-5 limit)
- ☒ `RecipeCard` chef overlay position is asserted, not just "it renders" — Phase 20 pins it to the cover's **bottom-right** (v2 moved it there)

Phase 19 — Authored recipe content, split from the demo seed

Separates the Secret Sauce Kitchen's **content** from the **demo fixtures** it was tangled with, so the fake chefs, taster accounts, and invented engagement numbers in `supabase/seed.sql` can be

deleted later without taking the real recipes with them.

The staged recipes arrived as one flat `recipeData/data.json` with a duplicate entry and nine content defects (B025) in a shape the app cannot use: `servings` as free text, `amount` as one opaque string ("1 1/4 cup" — the servings scaler multiplies a `numeric`, so it cannot scale a string), no ingredient/step groups, and every time field null.

Authoring format

- ✓ `recipeData/recipes/<slug>.json`, one recipe per file — the filename **is** the identity, so a duplicate slug is impossible by construction. Readable diffs; no merge conflicts when two branches each add a recipe.
- ✓ `recipeData/schema.json` — every field documented against the column it maps to. Deliberately **not** loaded at runtime (no JSON Schema dependency in the toolchain), so the rules are duplicated in `tool/recipes.dart` and both must change together.
- ✓ `recipeData/README.md` — workflow, and the three things that are easy to get wrong (decimal quantities, integer servings, unattended time as `duration_minutes`).
- ✓ All 9 recipes restructured: grouped ingredients and steps, decimal `quantity + unit + note + is_optional`, integer `servings`, real `prep_minutes / cook_minutes`, and per-step `duration_minutes / temperature / tip`. Verified no content lost — 109 ingredients in, 109 out, every title matched.
- ✓ `recipeData/data.json` deleted; it is fully superseded and still in git history.

Tooling

- ✓ `tool/recipes.dart` — `validate / gen / check`. Generates `supabase/seed_recipes.sql`, which is committed.
- ✓ `melos run recipes:validate | recipes:gen | recipes:check; melos run db:recipes; db:reset` extended to `drop → create → seed → recipes`.
- ✓ CI runs `recipes:check`. Nothing reads the JSON at runtime, so without it a stale generated `.sql` would be applied to a database and nobody would notice.
- ✓ Unused-ingredient lint (the margarita's orphaned orange liqueur, B025), suppressed on recipes with a catch-all step. Confirmed non-vacuous by re-introducing the defect.

seed_recipe_v2

- ✓ Group-aware helper, distinct name from `seed_recipe` so the two never overload each other. Numbers `step_order` **from 0 within each group**, matching `SupabaseRecipeRepository._persistContent` — continuous numbering would have been silently renumbered by the first edit.
- ✓ B024 guard: the drop block for historical signatures lives in the file that recreates the function, with the rule stated for the next signature change.
- ✓ `execute` revoked from `public / anon / authenticated` on both new helpers (B026).
- ✓ Demo ratings applied through `seed.sql`'s tasters via a `to_regprocedure` guard, so this file keeps working after that file is deleted.

Verification (local Postgres, 2026-08-18)

Ran in a throwaway database rather than the project's own stack — Docker was already serving an unrelated project on port 54322. Postgres 17.6 vs the config's 15; the `auth` and `storage` schemas were stubbed to the columns `0001_init.sql` touches. So this proves the SQL applies and the data lands; it does **not** exercise real auth, real Storage, or PostgREST.

- ☒ `0001_init.sql` → `seed_recipes.sql` applies clean; **twice** with no duplication (9 recipes / 109 ingredients / 41 steps either way)
- ☒ Both interleavings with `seed.sql` — 24 recipes, 24 distinct titles, both function overloads present exactly once, no error in either order
- ☒ Grouping and per-group `step_order` correct (teriyaki: sauce 0-1, skewers 0-3)
- ☒ `numeric` quantities, `note`, and `is_optional` all land correctly; `null` quantity for "to taste" survives
- ☒ `notes` appended to `description` as a trailing paragraph
- ☒ `proacl` on both helpers is `postgres=X/postgres` — PUBLIC/anon/authenticated revoked
- ☒ Kitchen `chef_score` unchanged at 10189 / `head_chef` (`public_recipe_count` 6 → 15), so the authored recipes add no phantom engagement

Migrating the six recipes already in the database

- ☒ `Classic Margherita Pizza`, `Spaghetti Aglio e Olio`, `Chicken Tikka Masala`, `Fluffy Buttermilk Pancakes`, `Fresh Guacamole`, `Brown Butter Chocolate Chip Cookies` authored into `recipeData/recipes/`. Source was `seed.sql`, which is what the hosted database contains — B022's audit established that all six still have exactly one `recipe_versions` row, so none has ever been edited through the editor.
- ☒ Titles kept byte-identical (the `(owner_id, title)` dedupe key), so re-applying against the hosted project is a no-op rather than a second copy.
- ☒ Engagement counters and taster ratings carried over as per-recipe `demo` blocks.
- ☒ The six `perform seed_recipe(...)` calls **removed from `seed.sql`** — one definition per recipe. `seed.sql` keeps the accounts, the `d1-d7` demo recipes, and the rating machinery.
- ☒ `config.toml` gained `db.seed.sql_paths = ["/seed.sql", "/seed_recipes.sql"]`, so `supabase db reset` still populates Discover in one command.
- ☒ Verified (re-run after `Easy Guacamole` was dropped): 23 recipes / 64 ratings / 16 profiles, both files applied twice, and the **identical** leaderboard — Kitchen 10189 `head_chef` with `public_recipe_count` 14, Amara 21000 `master_chef`, the Chen Wei / Greta Lindqvist `dense_rank` tie, Dara at exactly 100 `line_cook`.
- ☒ Verified the bad order too: recipes-first creates every recipe with **0** ratings and an explicit notice, and `seed.sql` → re-run `seed_recipes.sql` backfills to the identical 64 / 10189.

Two guacamoles, resolved. `Fresh Guacamole` (from the database) and `Easy Guacamole` (from the staged file) were different recipes — 3 avocados with coriander versus 2 with cherry tomatoes, jalapeño, and garlic. `Easy Guacamole` was dropped; `Fresh Guacamole` is the one that was already live. **Product rule confirmed in the process:** a chef may publish two recipes for the same dish, however similar, *provided the titles differ*. Duplicate content is allowed; duplicate titles are not, because `(owner_id, title)` is the import key — a second file with the same title silently collapses into one row rather than erroring, which is why `tool/recipes.dart` rejects it outright.

Deferred

- ☐ **recipes.notes column.** There is nowhere to put a recipe-level note, so `tool/recipes.dart` appends it to `description`. Lossless, but it lands in the card summary.
 - ☐ **Reverse-direction lint** — a step calling for an ingredient the list never mentions. Three of B025's nine defects were exactly that; it needs a lexicon and is still a manual read.
 - ☐ **No SQL test in CI**, as with every other `.sql` in this repo — `recipes:check` compares text, it never runs the generated SQL.
 - ☐ Retire `supabase/seed.sql` entirely once there is real traffic — the accounts, the `d1-d7` demo recipes, and the rating machinery. Recipes already survive without it; the only loss is the demo ratings, which is the point.
-

Phase 20 — RecipeCard v2

The card was redrawn in Claude Design ("RecipeCard v2", Secret Sauce project → [Design System.dc.html](#))

and ported as-is. Same inputs, same data, new anatomy: the **name leads the card as a banner** on `colorScheme.primary` instead of sitting under the photo, so it never competes with the cover and stays legible over a dark or busy image.

- ☒ Title banner (`primary / onPrimary`, 14 × 10 padding, `titleMedium` w700, **max 2 lines** then ellipsis) as the first child of the card
- ☒ Cover moved below the banner and made the only **flexible** child — a two-line title eats cover height, never card height
- ☒ Chef overlay moved **bottom-left** → **bottom-right** of the cover (same scrim, same `ChefBadge(compact: true, onSurfaceImage: true)`)
- ☒ Visibility chip moved into the banner and reduced to **icon-only** with the label as a `Tooltip` — a "Private" label is the first thing to overflow at 2.0× text scale
- ☒ Footer: description (2 lines) + a `outlineVariant` rule above the time / rating / difficulty row
- ☒ Fix B026 in that row: the difficulty badge had a flex of its own, so it reserved half the row whatever its label said — `4.9 (8)` truncated to `4...` at one-column widths. It is now the only non-flex child, capped at half the row (`LayoutBuilder + ConstrainedBox`; a bare intrinsic badge overflows by 1px at 276 / 2.0×), and sits flush right as the mockup has it
- ☒ **Fixed height, not fixed aspect:** `kRecipeCardHeight = 352` exported from `design_system`; `recipe_grid.dart` passes it as `mainAxisExtent` instead of `childAspectRatio: 0.82`, which is what left dead space under every card on a wide window
- ☒ The card owns its own height (`SizedBox`), so an unbounded-height parent can never leave the cover's `Expanded` unbounded (the original B001 shape)
- ☒ Tests: envelope suites (276 / 320 px × 1.0 / 2.0×) still green, `find.text('Private')` → `find.byTooltip('Private')`, plus new fixed-height and **overlay-is-bottom-right** assertions
- ☒ `melos run analyze + melos run test --no-select` clean (67 tests)

One thing from the mockup was **not** ported: the banner's serif (`Newsreader`) title. That is the design's separate open question — "typography is the one upgrade", a `google_fonts` dependency plus

a `textTheme` in `app_theme.dart` — and it applies to every title in the app, not just this card. The banner uses the theme's `titleMedium`, so it follows whatever that decision lands on.

Flowing grid (the design retired the old card and gave the new one a max width)

Claude Design deleted the previous card and added `.rcard { width: 100%; max-width: 340px }` — "width is fluid between **264px** and **340px**, so a wide grid adds a column instead of stretching three cards across 1400px".

- ☒ `FlowGridMetrics.fit` in `adaptive.dart`:
pure function of the available width → column count, tile width, and the gutter that keeps a capped row centred. No breakpoint involved, so a drag-resize reflows continuously
- ☒ `kRecipeCardMinWidth` (264 at the time; **288** since B048) / `kRecipeCardMaxWidth` (340) alongside `kRecipeCardHeight`
- ☒ `RecipeGrid` wraps its `GridView` in a `LayoutBuilder` and feeds the metrics in as `crossAxisCount` + extra horizontal padding — the delegate always divides the full extent between its columns, so **less width to divide** is the only way to cap a tile
- ☒ `responsiveColumns` stays for navigation chrome and non-card grids; the recipe grid no longer uses it (its doc comment now says so)
- ☒ Tests: `flow_grid_test.dart` sweeps every width from 264 to 2400 asserting the tile stays inside its bounds, the columns never decrease as the window widens, row + gutters account for every pixel, and no wider column count would have fit;
`apps/app/test/recipe_grid_test.dart` pumps the real grid at 390 / 700 / 1000 / 1440 / 2000, checks centring at a capped width, and resizes 400 → 1000 → 640 to prove it reflows
- ☒ Card envelope suites re-pinned from 276/320 to **264/340** — the grid's real extremes
- ☒ `melos run analyze` clean, 82 tests pass

Card revision (2026-08-20 — the design fixed two things the port inherited)

`Design System.dc.html` was updated after the port: the banner is now a fixed **65px** band with the title centred in it, and the fluid range starts at **288px**, not 264. Both are ported.

- ☒ Banner is a fixed band — `kRecipeCardBannerHeight` (65) × `context.textScale`, applied as a minimum with the title vertically centred, so one-line and two-line names give the same card and neighbouring covers start at the same y (B047). Scaled rather than pinned: two lines of 2.0× type do not fit in 65 raw pixels
- ☒ Band capped at `kRecipeCardBannerMaxScale` (2.0) — the card's height is fixed, so an unbounded band starves the cover: at 3.0× it took the card from a 17px overflow (the pre-existing B049 budget failure) to 48px. Clamped it is 13px, better than the baseline
- ☒ `kRecipeCardMinWidth` 264 → **288**, the floor at which the whole time / rating / difficulty row fits *uncut*; at 264 the grid was buying an extra column out of the footer (B048)
- ☒ Tests: new `title banner` group (same centre for one and two lines at 1.0× and 2.0×, clamp at two lines); envelope suites re-pinned to 264 (below the floor — must degrade) / 288 / 340; `flow_grid_test` and `recipe_grid_test` re-pinned to the new column counts (1408px is four capped cards, not five squeezed ones)
- ☒ `melos run analyze` SUCCESS, `melos run test --no-select` SUCCESS — 203 tests (core 42, design_system 91, app 70)

Deferred

- ☐ Typography upgrade (Newsreader + Manrope via `google_fonts`) — app-wide, needs a yes/no
- ☐ Cover photography is still placeholder in the design; real shots may change the 352 px height
- ☒ Screenshot pass on the revised card — **run 2026-08-25, and it found B080**,

which is the whole argument for having deferred it rather than dropped it. "The metadata row fits uncut at 288" was **false**: a rated recipe with a `Medium` badge printed `★ 5... (1)`, and on one card `1...` was really `1.5` — the ellipsis rounds a rating down to its first digit, on the one element the pill exists to show, while the *count* the code's own comment says yields first came through whole. 5 of 20 visible cards on a real grid at 1248px (4 columns × exactly 288). Two stacked `RenderFlex` mistakes, both Gotcha 21's shape one level deeper than B026 fixed it; both fixed, and re-shot to confirm.

What made it invisible: an ellipsis is not an overflow, so `takeException()` stayed null — the envelope suite's own comment had said as much and named this pass as the check. The regression test that now exists asks `RenderParagraph.didExceedMaxLines` instead of measuring pixels, so it states a *law* (`the rating value is the last thing the row gives up`) rather than a width, and survives the harness's fixed-width font

- ☐ **B049** — the card overflows at 3.0× text scale and always has. The fix is the spotlight card's shape (tile height computed from text scale, `mainAxisExtent` with it), which is a grid change; deliberately out of scope for the design port
- ☐ Nothing caps the grid's overall width, so a 4K window gets ~13 columns. If that reads as too many, the fix is a max content width on the screens, not on the card

Phase 21 — Top navigation v2 (web only)

Redrawn in Claude Design (`Design System.dc.html` → "Chrome & states") and ported. The old bar put brand, four destinations, and a `New recipe` button in one row and needed ~700px; it overlapped at medium. The design's fix is three changes, all implemented in `top_nav_bar.dart` — the row now needs ~470px.

- ☒ **Profile leaves the destination list** and becomes the avatar at the far right, where it also reads as *account*. `PopupMenuButton` behind it: name + `TierChip` header, Profile, Sign out. Nothing selected in the pill on `/profile` — the old shell highlighted Discover there, because `indexWhere` returning `-1` fell back to 0
- ☒ **Destinations sit in a centred segmented pill** (`surfaceContainerHigh` track, active chip lifted back to `surfaceContainerLowest` with the design's `0 1px 2px` shadow, `primary` w800 label + filled icon)
- ☒ **New recipe left the bar** for the page it belongs to: the My Recipes header (labelled `FilledButton` on web, the existing icon on compact, where the shell's FAB is the labelled call to action). Search already lives in Discover's search bar
- ☒ Signed out: `Sign in` + `Sign up` (`/auth?mode=signup` opens the screen on its sign-up side), collapsing to one filled `login` button at medium. My Recipes is not offered signed out — it could only bounce to `/auth`
- ☒ `ChefAvatar` in `design_system` (exported): initials fallback, optional `primary` ring and tier dot, so rank is readable at 34px. `ChefBadge`'s private `_Avatar` was folded into it

- ☒ `myProfileProvider` in `core/src/providers.dart` — the bar's avatar and the profile screen read one cached profile instead of two. `valueOrNull`, so a slow or failed read still leaves a usable account button rather than a spinner in the chrome
- ☒ `NavDestination` + the two lists in `nav_destinations.dart`: mobile keeps four slots including Profile (a phone has no room for an account menu, and a fixed four does not reflow on sign-in); web has Discover / Chefs / My Recipes-when-signed-in
- ☒ **Labels never wrap.** The pill measures its labels with a `TextPainter` at the live text scale and picks the most generous mode that fits: all labels (expanded), active label only (medium, and expanded when the measurement says so), icons only. The bar height itself scales with the text scaler, capped at 1.6×
- ☒ The pill is centred **on the bar**, not between the clusters — `CustomMultiChildLayout` reserves `max(brand, actions)` on both sides, so the brand's width does not push it right
- ☒ Tests: `apps/app/test/top_nav_bar_test.dart` — destinations per auth state, no Profile and no `New recipe` in the bar, the account menu opens, medium collapses to `login`, label degradation, **the pill is centred on the bar** (the one contract `_BarLayout` exists for), both auth doors, and a 600/760/1000/1400 × 1.0/2.0× no-overflow envelope; `apps/app/test/my_recipes_header_test.dart` covers the header's new labelled button
- ☒ `melos run analyze` clean; `melos run test --no-select` green (105 tests)

Review pass (`/code-review`, same change set). Fixed: the `TextPainter` in `_fit` ran inside `LayoutBuilder` and was never disposed — a paragraph leak per layout pass, i.e. per frame of a drag-resize; `TopNavBar.height` defaulted to `kTopNavHeight`, silently opting a caller out of the text scaling `heightFor` exists to provide, so it is now required; `_fit`'s fall-through to icons-only now asserts that icons-only actually fits, since below that the pill has nothing left to degrade; `ChefAvatar` added to SDS §8. **Refuted by measurement:** the labelled `New recipe` button was flagged as a B016-shaped overflow (it wants ~68px in a 56px toolbar at 2.0×) — probing it with the guard removed showed the toolbar clamps it with no `RenderFlex` error, so the label stays at every scale and the envelope test now pins that instead of a fix that was not needed.

Compact is untouched: the design is web-only, so `NavigationBar` + FAB still ship as they were.

Deferred

- [~] Every shell screen still draws **its own** `AppBar` under the new bar, so web shows two stacked bars. The design's screen mockups have one; folding the page title into the content is a per-screen change, not a nav change. **Discover is done** (Phase 26 — the masthead *is* the title); Chefs and My Recipes still stack
- ☐ The design's avatar is a real photo when the profile has one — `ChefAvatar` supports it, but no seeded profile carries an `avatar_url`, so initials are what actually render

Phase 22 — Chefs v2: podium board + expanded chef card

Redrawn in Claude Design (`Chefs.dc.html`, drafts `1b` collapsed / `1c` desktop expanded / `1d` mobile expanded). The board today lists four icon chips and a score; a chef cannot tell **why** they are where they are, and a row goes nowhere when tapped. Phase 22 makes the score legible and the row an entry point.

Four choices settled with the user before any code (the design left all four open):

Question	Decision
Collapsed card	1b "podium" — tier spine, medal glyph for the top 3, % to next tier under the score
Score breakdown	Show the multipliers (<code>likes × 3</code> , <code>saves × 5</code> , <code>views × 0.2</code>) — the reason to open it
Primary action	None. Follow does not exist; the top-recipe rows tap through to <code>/recipe/:id</code>
"Top recipes" ordering	New <code>chef_top_recipes</code> RPC — exact, reuses <code>chef_score()</code> as the source of truth

Schema (`0001_init.sql`, idempotent)

- ☒ `chef_top_recipes(p_chef uuid, p_limit int default 3) → setof recipes, stable`, invoker-rights, ordered by `chef_score(like_count, save_count, view_count) desc` with deterministic tiebreakers. Filters `visibility = 'public'` **explicitly** (same reason as `chefs_leaderboard`: under invoker rights a chef would otherwise see their own private recipes in their own dialog and read different numbers than everyone else)
- ☒ `grant execute ... to anon, authenticated` in the existing `pg_roles` guard block (B013) — the dialog is signed-out safe like the board
- ☒ B024 drop block for the signature, in the file that recreates it, and the matching line in `scripts/drop.sql`

core

- ☒ `ChefScoring` (`core/src/chef_scoring.dart`, plain Dart — no codegen): the **mirror** of `chef_score()` and `chef_tier_for()`. Weights 3 / 5 / 0.2, thresholds 100 / 1000 / 5000 / 20000, plus `pointsToNext` / `progressToNext` / `nextTier` / `unitsToNext` / `breakdown`. Exported from `core.dart`
- ☒ `core/src/formatting.dart` — `groupedCount` / `groupedScore`. Hand-rolled rather than adding `intl`: the only need is thousands separators and the product has no localization story. `ChefStanding.scoreLabel` now delegates here, so the board reads `21,000`, not `21000`
- ☒ `ChefStanding` gained `isPodium`, `tierProgress`, `nextTier`, `nextTierLabel`, `pointsToNextLabel` — all derived from the **score**, so a card cannot show "0 to go" beside the lower tier
- ☒ `ChefRepository.topRecipes(chefId, limit) → .rpc(...).select(kRecipeSelect)`, same shape as the Discover RPC calls
- ☒ `ChefRepository.chefCount()` — exact `count` over `profiles` where `public_recipe_count > 0`, for the dialog's "Rank 2 of 148". `limit(1)` keeps the body one row; `count` respects filters but ignores `limit`

design_system

- ☒ `ChefStandingCard` — draft 1b. 6px tier spine, `workspace_premium` / `military_tech` medal for ranks 1–3 (numeral + `rank` below that), `ChefAvatar` tinted to the tier, dense `TierChip`, the four icon-chip stats, score, and `34% to Master / top tier reached`. `onTap` required — an inert row is what this design answers. Compact drops the stat labels
- ☒ The spine is a `Positioned` child of a `Stack`, **not** a stretched `Row` child: the card lives in a `ListView`, so its height is unbounded at layout time and `CrossAxisAlignment.stretch` hands the spine `h=Infinity`. Caught by the envelope tests on the first run; `IntrinsicHeight` would have worked but costs a pass on all 50 rows
- ☒ The score is a `FittedBox`, not an ellipsis — a truncated number is worse than a small one
- ☒ `TierLadder` — the four thresholds as **evenly spaced anchors** (`0 / 1/3 / 2/3 / 1`) with the fill interpolating between them, `Line` / `Sous` / `Head` / `Master` beneath, current rung highlighted. A linear points axis would squash the bottom two tiers into the first 5% of the bar
- ☒ `ScoreContributionBar` — one labelled input row (`1,980 likes × 3 ... 5,940`) plus its bar
- ☒ All three exported from `design_system.dart` (gotcha 14)

app

- ☒ `chefDetailProvider(chefId)` family + `chefCountProvider` in `chefs_providers.dart`. The recipe read is caught and flagged, not thrown: a database without the new RPC loses one section instead of the whole dialog
- ☒ `chef_detail_sheet.dart` — one content widget, two presentations: `showDialog` capped at `1152 × 720` on `≥600px`, `showModalBottomSheet` (`isScrollControlled`, drag handle, fixed header, scrolling body) at 94% height on compact — the default 9/16 cap cuts the ladder off. Two columns on desktop, stacked on mobile
- ☒ `chefs_screen.dart` — board rebuilt on `ChefStandingCard`, row taps open the detail
- ☒ No footer action bar on mobile (1d draws Follow + Recipes there; both were cut)
- ☒ 1c's "Rank recomputes nightly. Last updated 4 hours ago" is **wrong about this build** — `on_recipe_stats_change` fires on every like, save, view and visibility flip — so the card says that instead of shipping the mockup's copy

Tests

- ☒ `packages/core/test/chef_scoring_test.dart` — weights and every threshold boundary, pinned against the SQL values so the mirror cannot drift silently
- ☒ `packages/core/test/chef_models_test.dart` — grouped `scoreLabel` and the new derived fields
- ☒ `packages/design_system/test/chef_standing_card_test.dart` — medal vs numeral, tier spine, dense labels, top-tier copy, tap target, and a `320 / 360 / 600 / 760 × 1.0 / 2.0×` envelope
- ☒ `packages/design_system/test/tier_ladder_test.dart` — the anchor mapping, pinned
- ☒ `apps/app/test/chefs_screen_test.dart` — extended: the multipliers render, the rank line uses the count, a tap opens a `Dialog` on desktop and a `BottomSheet` on a phone, the card survives a database with no `chef_top_recipes`, and no `Follow / View all` leaks in

Verified

- ☒ `melos run analyze` — **No issues found!** in all three packages
- ☒ `flutter test` per package — core 39, design_system 58, app 49 passed (146 total, all green)
- ☒ **SQL on a real Postgres.** `0001_init.sql` re-applied in place to the running local stack (`supabase_db_secret-sauce`) with `ON_ERROR_STOP=1`: no errors, `chef_top_recipes` created with `anon=X/postgres,authenticated=X/postgres`. As `anon` the RPC returns the Kitchen's three highest-contributing recipes in the right order (3,636 / 2,553 / 1,693 points). As the **private-only** chef `d6`, their own `chef_top_recipes` returns **0 rows** while they own a private recipe with 5,000 likes — the explicit `visibility` filter holds under invoker RLS
- ☒ **Screenshots of a release build** (`flutter build web --release` + static serve, per B028) at 1440×900 and 390×844, board and expanded card. Four bugs came out of them — **B029–B032** — none of which the widget tests could see: the tests never pump the shell, and an ellipsised name is not an overflow
- ☒ **Applied to the hosted project** (2026-08-19, no production yet). `0001_init.sql` piped in with `ON_ERROR_STOP=1`, exit 0. `chef_top_recipes` and `chefs_leaderboard` both carry `anon=X,authenticated=X,service_role=X`; as `anon` the RPC returns the Kitchen's top three (3,636.2 / 2,553 / 1,693). Re-screenshotted against it: Top recipes populates with rating pills, likes · saves, and contributed points on both desktop and mobile
- ☒ Getting there needed a workaround, recorded as **B033**: `psql` is not installed on this machine and the direct `db.<ref>.supabase.co` host is IPv6-only, which the Supabase Docker container cannot route. The Session pooler host (IPv4) through the container's `psql` works

Not verified

- ☐ Nothing here was exercised on a real phone; compact was checked at 390×844 in a browser

Deferred

- ☒ ~~No public chef page.~~ **Built in Phase 30** (2026-08-25), which also retired the dialog this phase built: a chef tap now navigates to `/chef/:id` and the "View all N recipes" hole is closed by the page's paged grid
- ☐ Pagination: the board is still one `limit: 50` page, though the RPC takes an offset
- ☐ "Joined " comes from `profiles.created_at`, which is the **profile** row's age, not the account's

Phase 23 — Chefs page v3 (web): hero, spotlight card, rails

Status: done, web only. Build order, traps and the full account in [EXECUTION-PLAN.md Phase 23](#).

Redrawn in Claude Design: `Chefs Page.dc.html` (page, expanded 1440 × 900) plus `Chefs.dc.html` drafts `1e` (spotlight card, desktop 400 × 560) and `1f` (mobile sheet — deferred).

`/chefs` was one 760px column. It answered who is ahead, and since Phase 22 why — but only one chef at a time, only on tap, only by all-time score. Nothing stated the population, nothing showed who is

moving, and there was nothing on the page a visitor would look at for pleasure. **Scope is web/expanded only; compact is byte-identical to Phase 22.**

Eight decisions taken before code; the four marked **owner** were the owner's call on the plan:

Mockup draws	Decision
4:3 portrait window	owner: leave it empty with the default avatar — <code>avatar_url</code> , else the monogram <code>ChefAvatar</code> . No chef portrait asset exists
SEASON 1 / S1 · 004/148	Drop the season — the serial is 004 / 148
RECOMPUTED 4H AGO	Wrong about this build, like 1c's "nightly" in Phase 22. Reads LIVE · UPDATES ON EVERY LIKE, SAVE AND VIEW
Time windows + Momentum tab	owner: not now. Controls render disabled with a tooltip , not hidden
—	Recorded for whoever builds them: windowed views must exclude anonymous rows or B012's inflation vector reopens
Full rails	owner: placeholder cards + TODO. Empty against today's seed; do not seed dated rows — the counter triggers would move every <code>chef_score</code>
Newsreader + mono	owner: keep the existing font; <code>TODO(fonts)</code> at each site
Follow / Share card	Cut, same as Phase 22

Schema

- ☒ **None.** The plan called for four RPCs and needed zero, which is the phase's most useful finding. The hero's tier tiles are five bounded exact counts over `profiles` (PostgreSQL cannot `group by`, and tallying client-side downloads a row per chef); the spotlight card's "move" row is the chef's strongest stat, already in the leaderboard payload via `ChefScoring.breakdown()`, so a ten-card rail costs **no** extra requests. Nothing to deploy — the page works against the hosted database as it stands

core

- ☒ `ChefRepository.tierCounts()` → `Map<ChefTier, int>`, five parallel `count(exact)` reads filtered `public_recipe_count > 0` so they sum to `chefCount()`
- ☒ `ChefTier.wireValue` — the PostgreSQL enum label, restated because `json_serializable` keeps its mapping private and a `.eq('chef_tier', ...)` filter needs the string without a decode. Pinned by a round-trip test, plus a distinctness check so two rungs cannot silently share a label
- ☒ `pluralNoun / countOf` in `formatting.dart`, hoisted out of the closure inside `ChefStandingCard._Stats` — B031 now has one place to be got wrong instead of four
- ☒ No `@freezed` field added, so no `build_runner` run was needed

design_system

- ☒ **ChefSpotlightCard** — built and reviewed first, standalone against draft 1e. Tier drives foil, rarity band, accent and glyph; the score sits where a trading card puts HP; the driver row names the input doing the most work with the arithmetic behind it; the footer is the tier-ladder bar. Portrait window per D1
- ☒ **spotlightCardHeight(context)** — the tile is fixed-size, so text growth has nowhere to go, and the intrinsic bands exceed the 356px budget well before 2.0×. Height is $356 + (\text{textScale} - 1) \times 168$; past 2.5× the growth clamps and the portrait absorbs the rest
- ☒ **SpotlightCardPlaceholder** — same frame and geometry, neutral bands, for the D6 shelves
- ☒ **CardRail** — icon tile, title/sub, 1-3 / 10, arrows dimming at the ends. A horizontal **ListView** + **animateTo**, not a translated transform: a web rail must answer drag and trackpad
- ☒ **ChefCardVariant.board** on **ChefStandingCard** — rank pill instead of a medal, no stat chips, a 3px tier progress bar on the bottom edge. A variant, not a second widget
- ☒ **context.textScale** in **adaptive.dart**, the one measurement three layouts now size from
- ☒ All exported from **design_system.dart** (Gotcha 14)

app

- ☒ **chefs_hero.dart** — dark brand gradient (literal colours in both themes, tier accents resolved at **Brightness.dark**), population pill, ranking rule, five tier tiles, window control
- ☒ **chefs_screen.dart** — **three** layouts: the untouched compact board; a single-scroll stacked page; and the two-column page (**Column(hero, Expanded(Row(panel 404, rails)))**) when the window is expanded **and** the text scale is at or below **maxTwoColumnTextScale**
- ☒ Deviates from the draft's page-scroll + **position: sticky** on purpose — the panel already owns a scroll container there, and nested pinned slivers are the fragile way to the same picture
- ☒ Board panel: **Score** / **Momentum** / **New**, and **Show all 148** wired to a wider first page — closing Phase 22's pagination item above
- ☒ Three rails; a spotlight card taps through to the existing expanded chef dialog
- ☒ **notYetTooltip** — explains a deliberately inert control and gets out of the way of a working one

Tests — 198 green, up from 146

- ☒ **chef_scoring_test.dart** / **chef_models_test.dart** — **pluralNoun** / **countOf**, and **wireValue** round-tripping through the decoder (core 42)
- ☒ **chef_spotlight_card_test.dart** — score/rank/serial/tier/totals, the driver row picking the real top contribution, **1 recipe** singular, top-tier copy, foil ramp, monogram fallback, the name-versus-score width ratio, and the tile envelope at 1.0 / 1.3 / 1.6 / 2.0 / **3.0**×
- ☒ **card_rail_test.dart** — arrows dim at both ends, position label arithmetic, no controls when the shelf fits, footnote
- ☒ **chef_standing_card_test.dart** — board variant: rank pill, no medal, progress value, **1 recipe**, tap target, 404px envelope at 1.0 / 1.5 / 2.0× (design_system 88)

- ☒ `chefs_screen_test.dart` — hero counts, three rails, placeholder footnotes, disabled controls staying inert, `Show all` widening the RPC's limit, an empty board showing **no** Popular shelf, spotlight → dialog, and the page envelope at 320 / 360 / 600 / 1000 / 1440 × 2.0×, which sweeps all three layouts (app **68**)

Verified

- ☒ `melos run analyze` — **No issues found!** ×3, `SUCCESS` read from the output rather than the exit code (B006/B007)
- ☒ `flutter test` per package — 42 / 88 / 68, all green
- ☒ `/code-review` against `CLAUDE.md` + the repo review checklist. Five findings, all fixed and pinned by tests in the same pass — **B037–B041**. Two of them (B038's 50/50 flex split, B041's empty tooltips) were review finds no test could make: neither overflows
- ☒ No SQL in the diff, so no local-stack run applies and nothing needs applying to the hosted project

Not verified

- ☒ **Screenshots** — **done 2026-08-22**, once Chrome was installed (this item's blocker). `/chefs` shot at 1400 against a `medium` sim through the B028 procedure: hero, five tier tiles summing to the 172 in the pill, the disabled window filter, the 404px board with **1 recipe** singular (B031) and grouped scores, three spotlight cards with serial/rank/driver row, and the two placeholder shelves holding their height. **No defects found** — the two the pass did find were on Discover (B059/B060, Phase 26)
- ☐ Nothing exercised on a real phone; compact is unchanged from Phase 22 either way

Deferred

- ☐ The windowed half: `chef_window_stats(p_since)`, `chefs_leaderboard_windowed(p_days, ...)`, the Trending and month rails, `Momentum`, and the hero's Month/Week. Wiring points carry `TODO(rails) / TODO(board) / TODO(hero)`; read the anonymous-views trap first
- ☐ Draft **1f** — spotlight card as a mobile sheet, with swipe and the Share/Recipes actions
- ☐ The **large** 400 × 560 spotlight size — nothing consumes it, and an unused variant is a second layout to keep correct for free
- ☐ The **New** sort would work today (`profiles.created_at` exists) but needs a column `chefs_leaderboard` does not return, so it moves with the rest of the windowed work
- ☐ Display/mono fonts — an app-wide decision, not a `/chefs` one
- ☒ ~~Still no public chef page, so a spotlight card's only destination is the expanded dialog.~~ **Phase 30** — a spotlight card now opens that chef's page, pinned by a test that asserts the *id*, not just that something opened

Phase 24 — Simulated population: a realistic user + engagement dataset

Status: in progress. Tooling and the first dish batch are built; the `sim` schema, the generator, and verification are not. Full design, distribution model, and the edge-case catalogue in

Every feature built since Phase 18 is a *ranking* of a population that does not exist. The database holds 21 accounts and 23 recipes, all of whose engagement counters were typed by hand into `seed.sql` / a `demo` block. Consequences, all of them already recorded elsewhere as limits:

- The leaderboard is 8 rows, so pagination, `dense_rank` at scale, and the hero's `004 / 148` serial are untested against real cardinality.
- `recipe_likes` / `recipe_saves` / `recipe_views` / `recipe_ratings` are **nearly empty** — the counters were written directly. Every windowed or dated query (the Trending rail, `Momentum`, the hero's Month/Week — Phase 23's whole deferred half) has nothing to read, which SDS §10.8 calls out as needing "its own answer before the rails do". This is that answer.
- Discover's Popular tab is a Bayesian average over ≤ 8 ratings per recipe; nothing proves the prior actually suppresses a one-rating recipe.
- No account in the database is a **non-creator**. The product's most common real user — someone who only ever reads recipes — has never been rendered.

The deliverable is a deterministic, idempotent, scale-parameterized generator that builds a plausible population *and its history*, with the engagement log as the source of truth and the denormalized counters derived from it (the reverse of how `seed.sql` works).

Content — the dish library

- [~] `simData/dishes/<slug>.json` — **25 of 120** authored dishes, same format as `recipeData/recipes/*.json` so a dish can be promoted into the curated set by moving the file. Written fresh, not copied (ingredient lists are not copyrightable; step prose is). Batch 1 was sequenced for **coverage before count** — all 7 targets below already pass at 25, so the remaining 95 add variety to an already-valid library rather than being load-bearing
- ☒ Coverage targets, asserted by `tool/sim.dart` over the whole directory: all 10 `category` values, ≥ 24 cuisines (25 dishes, 25 distinct cuisines), the full `difficulty` spread, a no-cook dish (`cook_minutes` 0), an overnight step (`duration_minutes` > 480), a multi-group dish (SDS §11.1), and a dish serving ≥ 8 . Warnings below 100 dishes, errors at or above it — a partial batch legitimately misses a category, a finished library does not
- ☐ `simData/people.json` — given/family name pools across ~15 locales, bio templates
- ☐ `simData/vocab.json` — the tag vocabulary (Zipf-weighted), title-variant templates
- ☒ `simData/README.md` + `simData/schema.json` — authoring workflow and the format delta from `recipeData` (no `demo` block; an optional `sim` block of `weight` + `variant_titles`)

Tooling

- ☒ Extract the validator out of `tool/recipes.dart` into `tool/recipe_format.dart` (a sibling, not `tool/lib/` — `tool/` is loose scripts, and a root `lib/` would have made the workspace package own it) so `recipeData` and `simData` cannot drift into two different definitions of a valid recipe. Proof the refactor is neutral: `recipes:check` still passes **byte-for-byte**

- ☒ `tool/sim.dart` (`validate` / `gen` / `check`) → `supabase/sim/1_sim_dishes.sql`, committed and CI-gated exactly like `seed_recipes.sql`. Creates its own schema and table so it is standalone; **upserts** by slug (a library should push content edits, unlike `seed_recipe_v2`) and deletes rows whose source file is gone
- ☒ `melos run sim:validate` / `sim:gen` / `sim:check`
- ☒ `tool/db.dart`: `db:sim`, `db:sim:verify`, `db:sim:clean`. `--preset` / `--seed` are written into `sim.config` before the generator reads them rather than passed as `psql -v` vars, so a hand-run file behaves identically to the wrapper. `db:sim:clean` deletes `auth.users` rows and requires an explicit `--yes`; every action prints the target host first (Gotcha 7)
- ☒ **db:reset DOES run the sim** — reversing the original plan, at the owner's request. Safe because `engage_existing` is false: the Kitchen and `d1-d7` counters stay byte-identical and every standing pinned in SDS §10.7 survives. Only the **rank**s move. Full reset from an empty database, sim included, is ~15s
- ☒ CI: `sim:check` added beside `recipes:check`

The `sim` schema (never `public`)

- ☒ `supabase/sim/0_sim_schema.sql` — `create schema if not exists sim`. Every helper function and registry table lives here, so PostgREST (which exposes `public` only) cannot reach them. This is B026's lesson applied by construction rather than by a `revoke` block
- ☒ `sim.rand(key, stream)` and the derived draws — `rand_normal` (Box-Muller), `rand_lognormal`, `rand_int`, `rand_bool`, `rand_ts`. All pure functions of `hashtextextended(key || stream, seed)`, **not** `setseed()` + `random()`: a hash is order-independent, so the same seed yields the same database regardless of plan or parallelism
- ☒ `sim.uid(kind, n)` — deterministic ids, so a re-run updates rather than duplicates. **bigint**, because at the `large` preset the composed `n * 10000 + ...` overflows a 32-bit int (B044)
- ☒ `sim.epoch_end()` — the time anchor, **pinned into** `sim.config` on first generate rather than read from `now()`. Determinism depends on it and the failure is silent; see B044
- ☒ `sim.actor` / `sim.recipe` registries — teardown deletes exactly what is listed here, never by an email or id pattern. The registry is the safety mechanism
- ☒ `sim.persona`, `sim.preset`, `sim.config`, `sim.title_variant` — the distribution is data, so retuning a share is a one-row edit, not a rewrite
- ☒ `sim.counter_baseline` — only used when `engage_existing` is on
- ☐ `sim.rand_zipf` — the tag vocabulary it was for is not built yet

Generation

- ☒ `supabase/sim/2_sim_generate.sql` — set-based, idempotent, one transaction, ~10s at `medium`
- ☒ **Personas**: ghost 22% / lurker 43% / collector 14% / casual 11% / regular 6% / power 2.5% / vault 1.5%. Measured at `medium`: 21.3 / 45.5 / 14.0 / 9.9 / 5.1 / 2.6 / 1.6, and **95%** of simulated accounts own no public recipe
- ☒ **Signups** on a compounding growth curve over 24 months

- ☒ **Recipes:** per-persona log-normal count; a dish from the library plus a deterministic variant. 1,671 recipes from 1,000 users at **medium**. Title uniqueness within an owner is guaranteed by a merged, **deduplicated** template sequence per dish — treating the dish's own variants and the generic pool as disjoint produced a real **(owner_id, title)** collision (caught by check D4)
- ☒ **Versions:** geometric edit count, 4,196 rows, **current_version_id** on the last
- ☒ **Forks:** ~4% of recipes, always from an older public recipe of a different owner
- ☒ **Engagement** as a funnel over a *view*, never independent: view → like → save → rate. At **medium**: 97,926 signed-in views, 19,930 anonymous, 6,483 likes, 4,898 saves, 1,839 ratings
- ☒ **Ratings** J-shaped, shifted per recipe by a latent quality term, plus a polarized set
- ☒ **Private recipes** receive engagement only from their **recipe_shares** rows (902 shares)
- ☒ Counters are **derived, not authored**: triggers disabled for the load, counters and **chef_score** / **chef_tier** recomputed set-based through the real **chef_score()** and **chef_tier_for()** (Gotcha 19), triggers restored
- ☒ **engage_existing** (default false) keeps every SDS §10.7 standing byte-identical
- ☒ Presets: **tiny** (60) · **small** (250) · **medium** (1,000, default) · **large** (8,000)

Verification

- ☒ **supabase/sim/3_sim_verify.sql** — 46 assertions that **raise exception** rather than print. This script is the test suite for this phase — there is no other coverage of the generator — and it found all three defects in B044 plus B045. Written when nothing in CI ran SQL at all; **OPT-T1 now runs it there** on a **tiny** population, which is what turned it from an opt-in script into a regression gate
- ☒ Counter invariants (A1–A8), including **view_count** excluding anonymous rows and repeat visits (B012) — plus A7/A8, which assert those rows **exist**, so A3 cannot pass vacuously
- ☒ Authorization invariants (B1–B5): no self-rating, half-star steps, no engagement on a private recipe from outside its share list, and no like or rating without a view
- ☒ Temporal invariants (C1–C6): nothing predates its parent, nothing is in the future, no fork older than its source
- ☒ Structural invariants (D1–D4): B022 ordering, every recipe has a **current_version_id**, no **(owner_id, title)** collisions
- ☒ Shape assertions (E1–E12) — persona coverage, heavy-tail concentration, J-shaped ratings, forks, long version histories, and the deliberate edge-case accounts. E3 and E9 are **population-aware**: both measure quantities whose ceiling is set by how many users exist, so at **tiny** they are relaxed or skipped **loudly** rather than tuned until they pass
- ☒ Idempotency: generate twice → identical counts and identical **sum(chef_score)**
- ☒ **d1–d7** + Kitchen standings asserted unchanged (F1–F3)
- ☒ Wall-clock: **medium** generate ~10s; full **db:reset** from an empty database ~15s
- ☐ RLS smoke test per persona with **set local role authenticated** — not written
- ☐ **large** preset never run; **master_chef** is asserted there but unverified
- ☒ **The assertion count was stale everywhere: 43 → 46.** 43 was right when Phase 24 shipped; Phase 26 added group **G** and no doc followed. Corrected in **CLAUDE.md**, this file, and **EXECUTION-PLAN.md** (nine call sites), and SDS §12 now says to recount rather than copy
- ☐ **CI's tiny preset does not run group G at all** — every check in it is gated on **v_users >= 250**, so **G3**, the fork-depth assertion that is the only guard on **sim.fork_bias** and therefore on **MOST FORKED** ranking anything, is exercised by **no**

automated run. ALL CHECKS PASSED at **tiny** means 43 of 46. Found while writing SDS §12 by running the verifier and reading the notices instead of the summary line. Options, none taken yet: raise CI to **small** (slower, but the shelf RPCs are what Phase 26 shipped), lower the gate with a preset-scaled threshold, or accept it and say so at the call site. Needs a decision, not a patch

Docs

- ☒ **docs/SDS.md** §12 — the simulation dataset: personas, distributions, invariants, and the derived-counter rule. **Written 2026-08-25**, in nine parts: the derived-counter rule and its three consequences, determinism (why hashing rather than **setseed()**, and the pinned time anchor), the persona and preset tables as data, the generator step by step, the nutrition draw, verification, safety, and the gaps. Two things the writing turned up, both now recorded there rather than only here — see the two items directly below
- ☒ **CLAUDE.md** — the new commands, and the two traps worth a gotcha entry: engagement rows must be loaded with triggers off (or the load is O(rows) profile recomputes), and teardown is registry-driven
- ☒ **README.md** — **db:sim** / **db:sim:clean** in the database section, with the B033 Docker **psql** form
- ☒ **docs/BUG-TRACKER.md** — B043 (tier calibration), B044 (three generator defects), B045 (first-apply ordering) recorded. Two more remain predicted, not yet observed: **recipes_search** recomputes **recipe_search_document()** per row for both the filter and the rank (no stored tsvector, no GIN index), and **can_read_recipe()** runs per row for every Discover read. Neither is visible at 23 recipes

Deferred

- ☐ **avatar_url** and **cover_image_url** stay **null** for every sim row. There is no image asset and an external URL 404s offline, which renders as a broken-image widget rather than the monogram fallback the null path gives. Config knobs exist for whoever wires a bucket
- ☒ A CI job that applies **0001_init.sql** → **seed** → **recipes** → sim **tiny** → **3_sim_verify.sql** — **done by OPT-T1** as **.github/workflows/database.yml**, which also walks the re-apply and upgrade paths. Not a "Postgres service" job as written here: **profiles.id** references **auth.users**, so it starts the real Supabase stack (database container only) instead
- ☐ Building Phase 23's deferred windowed half on top of this data — the rails now have rows to read, but they are still unwritten

Phase 25 — Restaurants & signature dishes (north star — designed, not started)

Design detail and prerequisites: [EXECUTION-PLAN.md Phase 25](#).

A restaurant is an **entity managed by profiles**, never a second principal: nobody signs in "as a restaurant", so auth, RLS, and the engagement model stay exactly as they are. Signature dishes are rows pointing at existing **recipes** — no second recipe system.

Prerequisites (each is real work, ordered)

- ☒ **Public chef page** (`/chef/:id`) — **done in Phase 30**, 2026-08-25. The restaurant page copies its shape: identity header (profile + optional standing) over a paged `RecipeAsyncSliverGrid`, on the root navigator, signed-out safe, no nav destination. Two things it learned that Phase 25 inherits — a page reached by URL needs its own single-row RPC (`chef_standing`, filtered *outside* the window), and a "belongs to this entity" recipe list must filter `visibility = 'public'` explicitly, because RLS hands an owner their own private rows and the page would then show them what nobody else can see.
- ☐ Decide the tier-calibration question (B043) before restaurant scores restate it — a restaurant aggregate over miscalibrated chef scores bakes the miscalibration in twice.
- ☐ SQL regression harness (Phase 11's item): Phase 25 adds tables, RLS, triggers, and RPCs on top of a schema whose existing SQL has no automated tests. Land the CI database job first so the new surface starts covered instead of joining the backlog.

Schema (`0001_init.sql`, idempotent, same rules as every prior phase)

- ☐ `restaurant_role` enum (`owner` | `chef`) — guarded creation like the other five enums
- ☐ `restaurants` table: `id`, `name`, `description`, `city`, `country`, `website`, `cover_image_url`, `created_by` → `profiles`, `created_at`, `updated_at`. Grants added explicitly (B013 — the blanket grant block runs before this table exists on upgraded DBs)
- ☐ `restaurant_members`: `restaurant_id`, `profile_id`, `role`, `title` (free text, e.g. "Head Chef"), `created_at`; PK (`restaurant_id`, `profile_id`). Association is optional by construction — a profile with zero rows here is the normal case
- ☐ `restaurant_signature_dishes`: `restaurant_id`, `recipe_id`, `sort_order`, `created_at`; PK (`restaurant_id`, `recipe_id`). `with check`: the recipe is **public** and owned by a member — a private recipe as a signature dish would leak its existence through a world-readable table
- ☐ RLS: restaurants + members + signature dishes world-readable (public directory); restaurant writes by `owner`-role members; membership writes by `owner` only; signature-dish writes by members. Every trigger that crosses ownership: `security definer set search_path` (B011 class)
- ☐ Optional denormalized `restaurants.member_count` / signature-dish aggregate — same recompute-from-scratch trigger pattern as `rating_*` and `chef_*`; decide the restaurant "score" formula only after B043 is settled
- ☐ `drop.sql` entries + B024 signature-drop blocks for any new helper

core / design_system / app

- ☐ `Restaurant`, `RestaurantMember` models (+ enums mirrored in `enums.dart` — update the "five enums" notes in `CLAUDE.md`/SDS when this lands)
- ☐ `RestaurantRepository` (abstract + Supabase impl, wired in `providers.dart`); embedding queries will need FK hints from day one — `restaurants` ↔ `profiles` is related two ways (`created_by` + members) at birth, the exact `PGRST201` shape Gotcha 17 documents
- ☐ `profiles`: no schema change — a chef's restaurant affiliation is read through `restaurant_members`, so the chef card/page gains an affiliation line without touching the

score machinery

- ☐ Widgets: restaurant card + member-chef row in **design_system** (barrel exports, envelope tests at 288px / 2.0× — same contract as every card)
- ☐ Routes: **/restaurants** (directory; signed-out safe) + **/restaurant/:id** (detail; signed-out safe). Nav destination decision per Gotcha 18: a fifth destination costs the web pill its labels — measure before adding, or reach restaurants through Discover/chef pages instead
- ☐ Editor surface for owners (create restaurant, manage members, pick signature dishes from the owner's public recipes)

Content / sim

- ☐ Extend the Phase 24 generator: a few % of simulated chefs belong to generated restaurants, signature dishes drawn from their public recipes — so the directory, empty states, and RLS paths are exercised at scale like everything else.

This is a prerequisite of the UI work, not a follow-up — no fixture today contains a restaurant, so the directory, the member list, and the signature-dish rail have nothing to render against until it lands. See the "Seed-data fit" gate in [CLAUDE.md](#)

Phase 26 — Discover v2: masthead, three shelves, one archive

Status: done. Build order and the reasoning behind each call in [EXECUTION-PLAN.md Phase 26](#).

Discover was **Popular / Trending / Recent** — one corpus ranked three ways, three times. Every tab answered "what is doing well", none answered "what am I in the mood for", and a visitor with no opinion about ranking had nothing to open. It now leads with three horizontal **shelves**, and the old three survive underneath as a **sort** on one browse grid, which is what they always were.

Three decisions taken before code; all three were the **owner's** call on the plan:

Question	Decision
Which three shelves	owner: 01 UNDER 30 (≤ 30 min), 02 WEEKEND PROJECTS (≥ 120 min or hard), 03 MOST FORKED . Time poles plus the one axis no other recipe app has
Shelves 2 and 3 are empty on a seed-only database	owner: fix it in the sim , not by authoring content. No new recipeData recipes; the fork weighting below is the fixture change
Do the tabs stay	owner: no. Masthead + shelves + one EVERYTHING ELSE grid with Top rated / Trending / Newest as a sort

Ranking — one signal per shelf, and that is the design

Three shelves ordered by the same key would be one shelf shown three times. **UNDER 30** ranks on the **Bayesian rating** (of the things you can cook tonight, the ones that turned out well),

WEEKEND PROJECTS on **saves** (you file a project away, you do not cook it tonight), **MOST FORKED** on **forks**. Each shelf prints its own rule — **RANKED BY SAVES** — the way the chefs hero states how the leaderboard is computed.

Schema — folded into `supabase/migrations/0001_init.sql`

Shipped first as `0002_discover_shelves.sql`, then **folded into the baseline on 2026-08-23** at the owner's call: the project is pre-release, nothing outside this machine depends on the schema, so a change belongs in 0001 as long as it stays idempotent. `0002` was deleted. The freeze resumes the day the schema reaches a database that is not ours — see [migrations/README.md](#) for why an edited baseline is silently wrong after that (the CLI never re-runs a recorded version).

- ☒ `recipes_quick`, `recipes_projects`, `recipes_most_forked` — `setof recipes, stable, invoker-rights, anon-callable, p_limit/p_offset`, every order ending `created_at desc, id` (Gotcha 24)
- ☒ `site_rating_prior()` — the `m = 5` phantom-ratings prior, hoisted out of `recipes_popular`'s inline CTE because `recipes_quick` needs the same arithmetic and a second copy of a ranking formula is Gotcha 19. `recipes_popular` is `create or replaced` to read it; the order it produces is unchanged. Called **once per query**, cross-joined — a per-row scalar would re-scan every public recipe for every public recipe
- ☒ `recipes_most_forked` counts **public** forks only. Not a filter — a correctness fix: these RPCs are invoker-rights, so an unqualified count is RLS-filtered and a private fork would count for its owner and nobody else, giving one recipe two different ranks
- ☒ Two partial indexes: `(prep_minutes + cook_minutes)` and the fork-source column, both `where visibility = 'public'`
- ☒ Grants to `anon`, `authenticated` in the file (B013), and every new signature listed in `scripts/drop.sql`

Sim — the fork tree gets a trunk

- ☒ A fork's source is drawn **weighted by reach**, not uniformly: `sim.exposure_draw` (the same per-recipe number \$7 turns into views) × the star-chef multiplier, raised to `sim.fork_bias()`. Measured at `medium`, same seed, same 74 forks: uniform spreads them over **54 sources, top recipe 2**; weighted lands them on **28 sources, top recipe 10** (then 5, 5, 3, 3, 3). Both fill the shelf — only one *orders* it
- ☒ `sim.exposure_draw()` in `0_sim_schema.sql` — one definition, because \$5 and \$7 must agree and they run at opposite ends of the generator
- ☒ `fork_bias` is a `sim.config` knob (default `2.0`), since it is the only thing deciding whether the shelf ranks at all
- ☒ `3_sim_verify.sql` section **G**: all three shelf RPCs are executed at any preset, their row counts asserted from `small` up, and the most-forked recipe must carry ≥ 3 forks — the assertion that fails if the weighting is ever flattened back
- ☒ Additive, so an already-generated database keeps its flat tree until `9_sim_teardown` + regenerate

core

- ☒ `DiscoverRepository.quick/projects/mostForked + publicCount()` (a HEAD request with an exact count — the masthead's one statistic, no rows over the wire)
- ☒ `RecordingHttpClient` takes response headers, so a `count()` request is testable at all

design_system

- ☒ `CardRailVariant.numbered` — set numeral, title in spaced caps, a hairline rule running out to the controls, ranking kicker. A **variant**, not a second widget (the `ChefCardVariant.board` precedent): the scroll controller and the `1-3 / 12` window are the substance and neither differs. The rule is the only flex child, so the title is capped rather than split 50/50 (B038)
- ☒ The header sheds its kicker, then its position label, then its arrows as it narrows — Discover's rails render at 320px, which the chefs rails never did
- ☒ `RecipeCardPlaceholder` — card geometry in neutral bands, so a loading shelf holds its height instead of dragging the shelves below it up the page

Claude Design (the mockups, 2026-08-23)

- ☒ `Discover.dc.html` redrawn in **both** projects — the bound `4e0d6b97` and `0611ea41` — so the tabs mockup no longer survives anywhere. Five frames: desktop full page, compact light, compact dark (doubling as the seed-only empty-shelf state), searching, and notes
- ☒ `tokens.css` now describes the **v2** card in both projects: `.rbanner`, the fixed 352px height, the 288/340 width bounds and the footer rule. A card is v2 when it carries an `.rbanner (:has())`, so the four canvases still drawn cover-first keep the pre-v2 geometry instead of being silently restyled — verified by re-rendering My Recipes, which is unchanged
- ☒ Side effect worth knowing: `4e0d6b97`'s tokens had no width bounds at all, so its other canvases now cap cards at 340px the way `0611ea41`'s already did — which is what the shipped grid actually does
- ☒ `My Recipes.dc.html` redrawn on the **v2 card** in both projects (2026-08-23): title banner first, cover, then the footer under a hairline; the visibility chip moved off the cover and into the banner, icon-only with the label as a tooltip, which is where the shipped card puts it. Its local `aspect-ratio: 0.82` is gone — that hack is what left dead space under every tile, and it would now fight the fixed 352px height
- ☒ Audit of the rest: **only that one canvas was ever pre-v2**. `Recipe Detail`, `Recipe Editor`, `Home Auth Profile`, `Chefs` and `Chefs Page` draw no recipe card at all (the `ph-img` in the chefs canvases is the spotlight card's *portrait* window), and `Design System.dc.html` already drew v2 — hand-rolled with inline styles, since the `.rbanner` class did not exist yet. An earlier note in this file said "four canvases"; that was wrong

app

- ☒ `discover_masthead.dart` — a printed masthead, not a second dark hero: rule, `THE PASS` kicker with the public-recipe count, title, one line of copy, search field on the title's baseline
- ☒ `discover_shelf.dart` — one widget configured three times; a loaded-empty shelf gives the **reason** ("no public recipe has been forked") rather than a spinner or a bare empty state

- ☒ `discover_screen.dart` — one `CustomScrollView`: masthead, shelves, **EVERYTHING ELSE**. Search replaces everything below the masthead. **No AppBar** — that closes the Phase 21 deferred item for this screen
- ☒ `SliverRecipeGrid` / `RecipeAsyncSliverGrid` — the grid and its loading/error/empty ladder as slivers, since a page that owns its scroll cannot nest a `CustomScrollView`. `RecipeGrid` and `RecipeAsyncGrid` are now thin wrappers, so the other five surfaces are untouched

Tests — 299 green, up from 281

- ☒ `discover_screen_test.dart` (**10**): the masthead's count, three shelves each tied to its own repository call, placeholders while loading, the reason strip when loaded-empty, a failed shelf retrying only itself, the sort switching the grid, search replacing the page, the envelope at 320 / 390 / 700 / 1400 × 2.0× with a non-vacuous presence check, and the two screenshot findings — the grid sharing the page's left edge (B059) and the sort link being the width of its own label with a drawn underline (B060)
- ☒ `card_rail_test.dart` (**+5**): numeral and caps title, the three-stage narrowing, the 320px envelope, and B057's over-long kicker (verified to fail without the fix — 721px overflow)
- ☒ `recipe_card_test.dart` (**+1**), `discover_repository_test.dart` (**+2**): each shelf calls its own RPC with the owner embed, and `publicCount` is a `HEAD` with `count=exact`

Verified

- ☒ `melos run analyze` — **No issues found!** ×3, after `melos run format` (safe since OPT-T4)
- ☒ `melos run test --no-select` — core 83, design_system 99, app 117, **SUCCESS**
- ☒ **The SQL ran, from an empty database.** After the fold (2026-08-23): `9_sim_tearardown` → `drop.sql` (0 tables left) → the folded `0001_init.sql` on a **clean schema** → seed → seed_recipes. 15 tables, 4 shelf functions, 2 shelf indexes. Then applied **0001 four more times**: zero errors, and `recipes_popular` still has exactly **one** overload, which is the B024 trap this file is most exposed to. All six Discover RPCs answer on seed-only data — quick **10**, projects **1**, most_forked **0**, popular 20, trending 20, search('lime') 2, prior m=5 / mean 4.578 — identical to what `0002` produced, so the fold changed no behaviour. This also closes the *fresh-apply* path, which the two-file version never exercised locally
- ☒ **The sim ran**, twice: once against `0002`, and again from scratch after the fold — 1,000 actors / 1,671 recipes, `3_sim_verify.sql` **ALL CHECKS PASSED** including \$G, and the fork tree reproduced **byte-for-byte** (74 forks over 28 sources, max 10), which is the same-seed determinism guarantee holding across a full rebuild. Fork concentration measured both ways at that seed: `fork_bias = 0` gives 74 forks over **54** sources with a maximum of **2**; `fork_bias = 2.0` gives the same 74 over **28** sources with a maximum of **10**. G3 (`>= 3`) fails the uniform draw, which is what it is for
- ☒ **Applied to the hosted project, 2026-08-23.** `0001_init.sql` piped through the DB container to the Session pooler (B033 form). Exit 0, no errors. Row counts identical before and after — 24 recipes / 22 public / 17 profiles / 26 versions / 64 ratings, 15 tables — so the re-apply cost data nothing. **Finding: hosted was well behind**, not merely missing the shelves; the apply also installed the `search_tsv` triggers (OPT-P1), `save_recipe` (OPT-A1)

and `recipe_versions_set_current` (OPT-S1), all of which reported `does not exist`, skipping on the way in

- ☒ **Verified as anon on hosted, which closes the signed-out half of the RLS gap.**
`set local role anon` inside a rolled-back transaction: all three shelves execute (so the grants landed), and anon sees **22** recipes where `postgres` sees 24 — RLS hiding the two private ones. The shelves return the same 10 / 1 / 0 for both roles, which is the "same rank for every caller" property `recipes_most_forked` was written for
- ☒ **End-to-end over PostgREST with the anon key** — the check no local stack can give: `recipes_quick` 200/10 rows, `recipes_projects` 200/1, `recipes_most_forked` 200/0, `recipes_popular` 200/20, `recipes_trending` 200/20. Supabase's DDL event trigger reloaded the schema cache by itself; no `notify pgrst` was needed
- ☒ **Screenshots** (Chrome installed 2026-08-22; B028 procedure — release build + `npx serve`): Discover at 390 / 700 / 1400 against the `medium` sim, `/chefs` at 1400, and a forked recipe detail at 1400. Two bugs, both fixed and pinned — **B059** and **B060**. Edges measured off the PNGs rather than eyeballed, which is what caught B059's 16-vs-32px
- ☒ `/code-review` against `CLAUDE.md` + the repo checklist. Two findings, both fixed in this change set — **B057** (the rail's kicker laid out unbounded, B039's class) and **B058** (a `drop.sql` comment claiming a dependency Postgres does not record through a quoted function body). Also caught a doc-sync miss: SDS §8's widget table now carries `RecipeCardPlaceholder` and `CardRail`'s two variants
- ☒ Two more from the screenshot pass, both invisible to every test because nothing overflows: **B059** (the archive grid inset 16 while the page was inset 32) and **B060** (the selected sort's underline was a width-less box — zero-width and undrawn on the web, full-width and stacking the links on a phone). Both fixed, both pinned by tests that fail without the fix

Not verified

- ☒ **Dark mode — verified 2026-08-23.** The browser cannot be switched after boot, so the app was: `themeMode: ThemeMode.dark` pinned in `main.dart`, rebuilt, shot at 1400 and 390, then reverted (`git diff` on `main.dart` clean) and rebuilt light. No defects — the shelf numerals take their scheme accents (`01` primary, `02` tertiary, `03` secondary) and stay legible, the card banner flips to the light-tone primary with dark `onPrimary` text, and the tier chip on the cover scrim holds its contrast (B055's fix works in both brightnesses)
- ☒ **A fresh apply** — done as part of the fold (see above): dropped schema → `0001` → seed → sim, all green
- ☒ **RLS.** The **anon** half is proven on hosted (above): shelves callable, private rows filtered, identical results to `postgres`. The signed-in `authenticated` half was bigger than this phase and predated it, so it went to the backlog as **BL-7 — now closed:** `supabase/tests/rls_matrix.sql`, 76 checks, in CI, and it found B061 on its first run
- ☐ Nothing exercised on a real phone; 390px is a resized desktop browser

Deferred

- ☐ Shelves do not page. A shelf is 12 rows and there is no "see all this shelf" route; the RPCs take `p_offset` already, so it is a route and a screen, not a query

- ☐ No shelf is personalised — nothing reads the signed-in user. **Under 30** is the same twelve recipes for everyone
- ☐ **MOST FORKED** shows no fork **count** on the card. It would need a denormalized `recipes.fork_count` (trigger + backfill + `kRecipeSelect` + the sim's counter pass), which is a schema change the shelf does not need to rank

Phase 27 — Recipe detail v2 (web): measured page, ingredients rail, method column

Status: done. Reading page (web + compact), cook mode, and the v1 layout retired. Drawn in the Claude Design canvas

`Recipe Detail v2.dc.html`; the as-built reference it was drawn against is `Recipe Detail.dc.html` (redrawn 2026-08-23 from real full-page captures).

The v1 page was one `Column` with 24px padding inside a `CustomScrollView` under a 240px `SliverAppBar`. On a 1440px window every ingredient line and every step ran the full 1392px measure, the metadata was a row of chips that said **70 min total** without saying which 70 minutes mattered, and nothing on the page could be *used* while cooking — no check-off, no progress, no sense of where you were.

What shipped (expanded windows only, $\geq 1000\text{px}$)

- ☒ **Measured page.** Content is capped at `kDetailPageWidth` (1140) and centred, so no line runs the window's width again. The header band spans full-bleed; the columns below sit inside the measure
- ☒ **Header band** carries identity: version line (doubles as the history opener), title at `displaySmall`, description, attribution, chef badge + rating, then the action row — Start cooking / Fork / like / save, and share + edit for the owner
- ☒ **Facts strip** replaces the chip row: Total · Hands on · Cook · Difficulty · **Longest wait** · Visibility as labelled cells. **Longest wait** is derived (the longest single step duration) and is the fact that decides whether a recipe is cookable tonight
- ☒ **Ingredients rail**, left, `kDetailRailWidth` (352) — reading order is gather → cook. Quantities sit in their own `kIngredientQuantityGutter` (86) column so the numbers scan vertically; scaled quantities turn **primary** when servings differ, times and temperatures never scale; names are sentence-cased at render (the DB stores them lowercase)
- ☒ **Check off ingredients and steps.** A checked ingredient strikes through and moves the **n of m gathered** counter; a done step collapses to one dim line with its duration, so the next thing to do is always the first full-size card
- ☒ **Step groups keep their identity** — group heading, step count, numbering restarting at 1 per group. The v1 layout rendered the same data; the redraw makes the grouping legible
- ☒ `formatMinutes()` in core (`1 h 10 m / 40 min / -`), with tests
- ☒ **Like/save extracted** to `LikeSaveButtons` in `detail_chips.dart` so the v1 body and the v2 band cannot drift apart on the toggle behaviour B051 fixed

Text-scale envelope — three overflows caught before they shipped

The new test file asserts *no exception* while pumping the whole page at {1000, 1440} × {1.0, 2.0}. It failed on the first run and found three real **RenderFlex** overflows, all the same shape (Gotcha 21: a non-flex child of a **Row** takes its intrinsic width, so a flexible sibling cannot save it):

Where	At 2.0×	Fix
Rail servings row	two IconButton s + count are wider than the rail	Wrap — the stepper drops to its own line
Rail footer	Clear checks alone exceeds the rail	Wrap — the note wraps under it
Cook-mode teaser	Start cooking is ~390px, wider than the 393px method column at 1000px	stacks above 1.3× , the shape /chefs uses

The rail width and the quantity gutter are now **bounded against text scale** rather than fixed (`context.textScale.clamp(1.0, kDetailRailMaxScale)`) — Gotcha 22.

Two review findings — both invisible on local fixtures

/code-review found two things no test and no screenshot on this machine could have surfaced, because the fixtures happen not to reach either state. Both fixed:

- **B065 (high, perf):** the header band watches **recipeVersionsProvider**, and **versions()** was a bare **.select()** — so every page open downloaded **recipe_versions.content_snapshot**, a whole recipe as **jsonb** (~10 KB × up to nine versions), to render a version **count**. Invisible locally because every seeded snapshot is **'{}'**. New **kRecipeVersionSelect** names the seven columns the client reads
- **B066 (medium):** an ingredient with a unit and **no** quantity rendered — and dropped the unit — reachable by typing **1/2** in the editor's quantity field, and the v1 renderer prints the unit in the same case, so the row read differently either side of the 1000px branch. The gutter now falls back **quantity + unit** → **unit** → **note** → **—**, and the note rides beside the name unless it took the gutter, so no combination loses a half

Seed-data fit

Existing fixtures cover both the reading page and cook mode, and this was checked before building each. Cook mode in particular needs a recipe with **more than one step group** (or the weighted progress bar and the per-group numbering are untestable), steps carrying **duration_minutes** (or there is no timer to start), and step prose that actually names its ingredients (or the derived "you'll need" list is empty everywhere). **Spring Vegetable Tart** has all three: 3 step groups, durations of 60 / 20 / 12 / 2 / 35 minutes including the 1-hour chill, tips, temperatures, and prose that says "the flour", "the onion", "the leek". No new fixture data was needed for either half, so none was added.

The rest, as checked for the reading page: **Spring Vegetable Tart** (**recipeData/recipes/spring-vegetable-tart.json**) has 3 ingredient groups and 3 step groups with

per-group numbering, an attribution, and zero engagement; **Chicken Tikka Masala** has ratings, likes, saves, temperature and tip fields on its steps. No new fixture data was needed, so none was added. What the local fixtures **cannot** show: cover images (no seeded recipe carries one, so the band's cover column is dead on the local stack — it is exercised on hosted only), and a recipe with `forked_from_recipe_id` set (the fork line renders from the model but no seeded recipe is a fork).

Cook mode — built (canvas frames C, D, E, H)

`/recipe/:id/cook`, its own route on the root navigator, **always dark** (the phone is propped under kitchen lights, so the mode overrides the theme — the only screen that does). Signed-out safe.

- ☒ **The step walk.** `flattenCookSteps` turns grouped steps into a flat list that still knows its group, so the header says `Filling · step 1 of 3` and numbering restarts per group while `Step 5 of 9` counts overall
- ☒ **Segmented progress, weighted by step count** — a 4-step crust and a 2-step bake are not halves of the same job. Fill counts steps *behind* the cook, so landing on a group's first step fills nothing
- ☒ **Step timers, several at once, one shared tick.** Start / pause / resume / +1 min / reset, seeded from `steps.duration_minutes` (a step without one gets no timer rather than a made-up default). A timer keeps counting when the cook moves on — the whole reason a step timer beats a kitchen timer — and the alarm is **state, not an event**, so a bake that finishes while you are on step 3 is still ringing when you look up
- ☒ **Finish screen** (frame E): steps cooked, wall-clock elapsed against the recipe's estimate, and the rating — asked at the one moment the cook knows the answer. Fork for non-owners, "not done — back to the last step" so it is a session state and not a dead end
- ☒ **Web layout** (frame H): step text at `displaySmall`, the ring beside the controls rather than under them, and a rail holding this step's ingredients and what is coming up. Space advances, arrows move, escape leaves
- ☒ **"You'll need" per step**, derived — see the honesty note below
- ☒ Both layouts pumped at 390 / 1000 / 1440 × {1.0, 2.0}; three bugs caught pre-ship (**B067–B069**), of which B067 is a new mechanism and is now Gotcha 25

Deferred by the owner's call (2026-08-23), and the copy says so. The canvas promises "screen stays awake" and "alarm rings even with the screen off". Both need plugins (`wakeLock_plus`, `flutter_local_notifications`) plus Android/iOS config on the committed runners, and neither is verifiable by a widget test on this machine. So the chime is Flutter's own `SystemSound` + `HapticFeedback` — real, and **foreground-only** — and the chips read "Keep this screen open" / "Chime when a timer ends". Adding either plugin means changing that copy in the same commit.

- ☐ **Screen-awake while cooking** (`wakeLock_plus`) — one dependency, no manifest edits
- ☐ **Alarm with the app backgrounded** (`flutter_local_notifications` + notification channel, `POST_NOTIFICATIONS`, iOS permission, exact-alarm handling)
- ☐ **The cook's note.** Frame E draws "add a note for next time"; `recipe_ratings` holds a rating and two timestamps and nothing else, so the field is **not drawn** rather than drawn dead. It needs a `note_text` column plus its grant, its RLS `with check`, a check in `rls_matrix.sql` (Gotcha 15), the model and the repository — its own change set

- ❑ **A real `step_ingredients` link.** `stepIngredients()` derives the per-step list by matching a distinctive word of each ingredient name against the step's prose (whole-word, stop-word filtered), because **no schema link exists**. It works on real recipe prose — "boneless chicken thigh" is found by "the chicken" — and it is a hint, not a checklist: a step that names nothing hides the panel, and "add the remaining spices" finds nothing. Promoting it to a checklist needs the table

Compact v2 — built, and v1 is gone (canvas frame B + frame F)

- ☑ **`recipe_detail_compact.dart` replaced the v1 hero**, rather than sitting beside it. The 240px `SliverAppBar` over one padded `Column`, and `recipe_content_views.dart` with it, are **deleted** — so the page below 1000px is no longer a different *design* from the page above it, only a different layout of the same one
- ☑ **It serves compact and medium both.** The canvas draws no medium screen, and a single-column cover-first page reads correctly at 800px. Keeping v1 alive for the 600–1000 band would have meant maintaining a third layout for a width nobody designed
- ☑ **Cover-first** with the chrome floating on it (back / history / share / edit), a scrim behind each button so a themed icon colour is never painted onto an unknown photo (the B055 mistake), and a `Private` badge on the cover instead of a facts cell. No cover → the same band, shorter, in `surfaceContainerHighest` — which is what the local stack always shows, since no seeded recipe carries one
- ☑ **Facts quad:** `FactsStrip(quad: true)` — Total / Hands on / Difficulty / Longest wait as 2×2. Six cells across 390px is 65px each, narrower than the word "Difficulty"
- ☑ **Pinned jump bar** — Ingredients / Method / Fork. Its content scrolls **horizontally**: a pinned sliver has one fixed height, so a `Wrap` cannot save it and a `Row` of intrinsic chips is the Gotcha 21 overflow waiting to happen at 2.0×
- ☑ **Ready to cook? pinned to the bottom**, outside the scroll as `Column(Expanded(scroll), bar)` rather than a `Stack` with a reserved padding — the bar's height grows with text scale, so any reserve constant is wrong at some scale
- ☑ **The two panels are reused, not reimplemented.** `IngredientRail(bordered: false)` and `MethodColumn` are the same widgets the expanded page uses; only the card border differs. That is deliberate — B066 was two copies of the ingredient list disagreeing
- ☑ Envelope at 390 / 600 / 800 × {1.0, 2.0}; it found **B070** on the first run

Still not built — the rest of the canvas

- ❑ **Sticky ingredients rail.** The canvas has it `position: sticky`; the Flutter page scrolls it with the content. Needs real sliver work (a pinned sliver beside a scrolling one), not a widget swap
- ❑ **Version history v2** (canvas frame G): what changed, not only when — needs a diff between snapshots, and a fork count the schema does not denormalize
- ❑ **Owner-fork lineage line** ("Forked from X · v3") shows only `Forked recipe`; naming the parent needs a second read
- ❑ Checks are session state (`checkedIngredientsProvider` / `doneStepsProvider`), not persisted. The copy says so — "Checks last until you close the app" — rather than promising the device

Phase 28 — Nutrition facts: per-serving label, rail tabs, manual entry (done)

Status: DONE (2026-08-24). Everything below shipped as planned; the three Deferred items at the bottom are the only open work. Three things the plan did not anticipate, all recorded where they belong: `Recipe.toJson()` needed an explicit `@JsonKey(toJson:)` for the nested model (**B071** — `explicitToJson` is off for this package and every other nested field on `Recipe` is `includeToJson: false`, so `nutrition` was the first one that had to flatten itself); the editor's collapse had to keep its fields registered with the `Form` (**B072**, `Visibility(maintainState: true)`) or a half-typed number could be hidden and then silently dropped; and hoisting the servings stepper out of `IngredientRail` restated its "is this scaled?" test instead of moving it, dropping the `servings == 0` guard (**B073**, found by `/code-review` — an extraction that restates a predicate is where a pure refactor stops being one). Verified on the local stack across the fresh path, the upgrade path, and `db:rls` at **79 checks** (76 + 3), with the grant check confirmed non-vacuous by reverting it once — and B072's test likewise, by deleting `maintainState: true` and watching it go red.

Recipes gain an optional **nutrition facts** panel

drawn like the label on a store product — the bold `Nutrition Facts` header, servings line, oversized Calories row, per-nutrient rows with a `% Daily Value` column, heavy rules between sections. It shares the rail with ingredients as **two tabs under the servings stepper** — `Ingredients` (the default) and `Nutrition` — on **both** detail layouts, and both tabs read the same `selectedServingsProvider`, because two surfaces printing different numbers for one serving count is the B066 class of bug.

Three input modes were named in the ask; two are in scope now: **manual entry** in the editor, and **leave empty** — a recipe without data shows `No nutrition info available` inside the tab. **Auto-calculate is deferred**, and no inert button ships for it (the Phase 27 rule: a drawn-but-dead affordance is worse than absence). Mechanism, alternatives, and order of work: [EXECUTION-PLAN.md Phase 28](#).

Decisions (taken in planning, before code)

- **Storage is one nullable jsonb column, `recipes.nutrition`** — not eleven numeric columns. The writable-recipe-column set is restated in ~13 places (both grant lists, `_writablePayload`, `save_recipe`'s two branches, `fork_recipe`, `_kRecipeColumns`, the model, `seed_recipe_v2` + its generator + its revoke/drop strings, the validator, `schema.json`, the read-side test pins); one column costs each copy one line, eleven would cost eleven each. `null` means "no info"; an all-empty entry is normalized to `null` **before** the repository so the empty state has exactly one representation. The key set *inside* the json is fixed — 11 label fields, all optional non-negative numbers — enforced by the model, the editor, and the authoring validator.
- **Values are per serving at the recipe's base `servings`, and the label never multiplies them.** Scaling 4 → 8 doubles the ingredients *and* the servings, so each serving is unchanged; a label that multiplied per-serving values by the factor would be wrong at every factor ≠ 1.

The stepper still visibly moves the label: the servings line prints the **scaled** count and a batch line (`8 servings · 1,920 kcal total`) is calories × scaled servings. That is what "depends on the serving size" means here.

- **The servings stepper is hoisted out of IngredientRail into the shared tab host**, so it stays on screen on either tab — nutrition depends on it exactly as ingredients do, and a stepper hidden behind the other tab would make the batch line unexplainable. The hoist re-opens the rail's width envelope (Gotcha 26) and the plan budgets a full re-run for it.
- **Tabs are two ChoiceChips in a Wrap**, not a SegmentedButton: the compact content box is 358 px and a segmented control is one intrinsic Row that cannot reflow at 2.0× (Gotcha 21). Tab state is a `StateProvider.autoDispose.family<RailTab, String>` beside `selectedServingsProvider` — `autoDispose`, so every visit starts on Ingredients.
- **The label widget is NutritionFactsLabel in design_system** (which already depends on `core`, so it takes `RecipeNutrition` directly), exported from the barrel (Gotcha 14), themed with `onSurface` tokens rather than literal black so dark mode holds. Rows with no value are omitted; `% Daily Value` is computed in core from the FDA daily-value constants.

Schema (`0001_init.sql`, idempotent — pre-release, so folded into the baseline)

- ☒ `alter table recipes add column if not exists nutrition jsonb` (nullable)
- ☒ Named check `recipes_nutrition_is_object` — `nutrition is null or jsonb_typeof(nutrition) = 'object'` — added via the guarded `do $$ ... pg_constraint` pattern the deferred FKs use
- ☒ `nutrition` in **both** column-level grant lists (insert + update). The RPC save path is `security definer` and would not catch the omission — the grant is what keeps a direct `PATCH` of a client-writable column working and the B050 story consistent
- ☒ `save_recipe`: insert-branch column + extraction, update-branch assignment — both as `nullif(p_payload -> 'nutrition', 'null'::jsonb)`. The arrow is `->` (jsonb), not `->>` (text — no implicit cast, runtime error); the `nullif` is because a Dart map with a null value arrives as **JSON null**, which is not SQL `NULL` and would fail the `typeof` check
- ☒ `fork_recipe`: `nutrition` added to its insert list — a fork is a deep copy and carries the label
- ☒ `recipe_snapshot` needs **nothing** — it is `to_jsonb(r) - 'search_tsv'`, so version snapshots pick the column up automatically
- ☒ `rls_matrix.sql`: a positive owner-update check on `nutrition` (the missing-grant failure is silent on the RPC path, so the matrix is where it becomes visible), plus the `save_recipe` round-trip literal extended to write a nutrition object and read it back, and one check that a JSON-null payload lands as SQL `NULL` (the `nullif` proof)

core

- ☒ `RecipeNutrition` frozen model (`src/models/recipe_nutrition.dart`): `calories`, `total_fat_g`, `saturated_fat_g`, `trans_fat_g`, `cholesterol_mg`, `sodium_mg`, `total_carbs_g`, `dietary_fiber_g`, `total_sugars_g`, `added_sugars_g`, `protein_g` — all `double?` (`numeric` arrives int-or-double, Gotcha 12), `includeIfNull: false` on `toJson` so stored json carries only entered fields, plus an `isEmpty` getter. Exported from

`core.dart` — the barrel is the only public surface, and both `design_system` and the app need the type

- ☒ `Recipe.nutrition` (`RecipeNutrition?`, wire key `nutrition`) + `melos run build_runner -no-select`
- ☒ `_kRecipeColumns` + `nutrition` (24 → 25) — the read-side twin obligation (Gotcha 17); without it the column decodes as null with no error
- ☒ `_writablePayload` + `'nutrition': recipe.nutrition?.toJson()` (13 → 14 keys)
- ☒ `% Daily Value`: FDA 2,000-kcal daily-value constants + `percentDailyValue()` + a value formatter that trims like `_trimQuantity`, in `core/src/formatting.dart` (or a sibling `nutrition_facts.dart`) — pure, tested
- ☒ Tests: decode (int / double / absent / JSON null), `toJson` omits null keys, the `chef_models_test.dart` select pin gains `nutrition` (it pins **membership**, so a new column passes silently unless added there), `recipe_repository_test.dart` fixture row + a save assertion on `p_payload['nutrition']`

design_system

- ☒ `NutritionFactsLabel` (`src/widgets/nutrition_facts_label.dart`): store-label design — heavy outer border, `Nutrition Facts` header, servings + per-serving lines, thick section rules, oversized Calories row, right-aligned `% DV` column, indented sub-rows (saturated / trans under fat; fiber / sugars / added under carbs), the 2,000-calorie footnote. Only rows with values render; no hard-coded black anywhere
- ☒ Barrel export (Gotcha 14) + widget test: row omission, %DV text, envelope at $\{320, 358, 493\} \times \{1.0, 2.0\}$ — the widths the two layouts actually hand the rail

app — recipe detail

- ☒ `servings_row.dart`: stepper + `Scaled from N` banner extracted from `IngredientRail` (which keeps its heading, list, and footer)
- ☒ `rail_panel.dart`: the shared host — bordered card on expanded, bare on compact (the `bordered` param moves here from `IngredientRail`); children: `ServingsRow` → tab chips → active pane
- ☒ `nutrition_tab.dart`: watches `selectedServingsProvider(recipe.id)`, renders `NutritionFactsLabel` with the scaled count + batch line, or the `No nutrition info available` empty state
- ☒ `railTabProvider` beside the other detail providers; compact's `_jumpToIngredients` also resets it, so the jump chip never lands on a hidden ingredient list
- ☒ Both layouts swap `IngredientRail` for the host; `IngredientRail` loses its `bordered` param (the container moves to the host), so the compact suite's `.bordered isFalse` assertion moves with it — an expected API break, not a regression. Cook mode untouched — it has its own gutter and already reads the same servings provider
- ☒ Tests in both suites: Ingredients is default, switch shows the label, empty recipe shows the empty text, the stepper moves the batch line, jump-chip reset; envelope matrices re-run **per tab** (compact 390 / 600 / 800, expanded 1000 / 1440, $\times \{1.0, 2.0\}$) — the rail restructure re-opens B070's envelope (Gotcha 26)

app — recipe editor

- ☒ `EditNutrition` in `edit_models.dart`: 11 controllers, `fromModel` / `toModel` (all-empty → `null`, never `{}`), dispose wired into the screen's controller-dispose list — a field the draft drops is a field the next save deletes (B035 / Gotcha 20)
- ☒ `nutrition_editor.dart` panel (same contract as the other editors: draft + `onChanged`), after Attribution; collapsed when empty, expanded when values exist; numeric `TextFormField`s with inline validators so a non-parseable entry **blocks save** instead of silently dropping (the B066 lesson)
- ☒ Round-trip tests: every field survives load → save, all-empty → `null`; editor envelope re-run with the panel expanded (320 / 360 / 600 × 2.0×)

Content & seed-data fit (the gate: data extended in this change set)

- ☒ `recipeData/schema.json`: optional `nutrition` object (`additionalProperties: false`, 11 non-negative numbers). `tool/recipe_format.dart`: `nutrition` in `_baseRecipeKeys` (optional, never required) + a `_validateNutrition` (unknown keys stay hard errors). `simData` inherits via `$ref`; dishes stay nutrition-free, because a label belongs to a recipe as published rather than to the dish idea, and hand-authoring 120 of them would be busywork with no signal in it
- ☒ `seed_recipe_v2` gains `p_nutrition jsonb default null, appended last` — a signature change, so the previous 17-arg signature joins the in-file `drop function if exists` list (B024 / Gotcha 5) and the revoke strings and `drop.sql` line move with it; all of it emitted from `tool/recipes.dart`
- ☒ Dummy values, per the ask: `chicken-tikka-masala` and `spring-vegetable-tart` get all 11 fields = 10; the other 12 files get an explicit `"nutrition": null` (the format spells optional fields out) and exercise the empty state. `melos run recipes:gen`, commit both; CI `recipes:check` guards staleness
- ☒ What the fixtures **cannot** show: hosted keeps `nutrition = null` — `seed_recipe_v2` early-returns on an existing (`owner_id, title`), so a re-seed never pushes the dummy 10s to production. That is the intended outcome (placeholder data must not ship); real label values are the content task in Deferred. (*Superseded 2026-08-25: hosted was in fact still on the Phase 27 schema with no `nutrition` column at all, so the 10s could not have reached it by any route. It now carries 29d's real labels — see the BL-5 rollout bullet.*)
- ☒ **Sim labels (added 2026-08-24, owner's ask)**. The two all-10 fixtures make the panel *reachable*; they do not make it look like a product — every `% Daily Value` they print is nonsense and there are two of them. So `sim.nutrition_for(key, category)` (`0_sim_schema.sql`) draws a label per simulated recipe and `2_sim_generate.sql` writes it, giving ~1,320 varied labels across the population. Three properties make it worth looking at rather than merely non-null: values are **derived from a calorie draw**, not field by field, so the label adds up at 9/4/4 kcal per gram; ranges are **per category**, from the `sim.nutrition_profile` table (Dessert draws high sugar, Main high protein, Sauce low everything); and every sub-value is **bounded by its parent**. ~20% get no label, because the empty state is a real state and a population where every recipe has one cannot demonstrate it. Randomness is `sim.rand` only — same seed → same labels (B044), verified by regenerating twice and comparing an md5 over all 1,671 rows. New assertions **D5–D8** in `3_sim_verify.sql` (39 → 43): self-consistency, the exact 11-key set, non-negative numbers, and that both the labelled and unlabelled states actually exist

Verification plan

- ☒ `melos run analyze · melos run test --no-select · melos run format`
- ☒ Local stack: `supabase db reset`; then the Gotcha 6 **upgrade path** — old schema + old seed (via `git show`) with the new files layered on top. The `seed_recipe_v2` signature change is exactly the class that ships green through the two easy paths (B024)
- ☒ `melos run db:rls` — the count moves from 76 with the three new checks
- ☒ `melos run recipes:check · melos run sim:check`
- ☒ Screenshots (B028 procedure): both tabs × both layouts × {empty, populated} × {light, dark} — dark is where a hard-coded label black would betray itself
- ☒ Docs on completion: SDS §3.2 (recipes table), §7.1 (the tab host), §11 (authoring format); CLAUDE.md feature-map row for `/recipe/:id` and the server-owned/writable column lists

Deferred

- ☐ **Auto-calculate from ingredients** — now **Phase 29** (planned 2026-08-24, not started): a committed USDA FoodData Central registry, ingredient-level food links, estimated labels with an **Estimated** disclosure. The schema was already shaped for it — it writes the same column
- ☒ **Real nutrition values** for the 14 authored recipes — done in Phase 29d: the estimator's own output replaced the dummy 10s, 12 recipes **auto**, one manual, one null
- ☐ Micronutrients (vitamin D, calcium, iron, potassium — the label's lower block) and a per-100 g display — stays deferred; Phase 29 keeps the 11-field key set

Phase 29 — Auto nutrition: food registry, ingredient links, estimated labels (DONE — 2026-08-25)

Phase 28 shipped the label and manual entry; this phase makes the label **computable from the ingredients**, so most cooks never type eleven numbers. The editor's nutrition panel becomes a three-way choice — **Automatic / Manual / None** — where Automatic estimates the per-serving label from the ingredient list and marks it as an estimate. Mechanism, alternatives beaten, and the traps: [EXECUTION-PLAN.md Phase 29](#).

The data source is the USDA **FoodData Central** CSV bundle (2026-04-30 release, public domain), **measured before this plan was written** (2026-08-24, on the actual bundle): 9 of the 11 label fields have ≥ 94% coverage across the 13,694 generic foods; **added sugars has zero rows** in every generic food and must be rule-derived; trans fat is 31% and stays null-heavy; total sugars is nutrient id **2000**, not the deprecated 1063. The 3.1 GB bundle is an **authoring-time input only** — nothing in the repo, CI, or the app ever reads it.

Decisions (taken in planning, before code)

- **Ingredients are linked to foods at input time, not matched at save time.** A typeahead in the ingredients editor sets an invisible `food_id` when the cook picks a suggestion; free text past the dropdown is always allowed and never blocked. Matching-by-inference survives only as a one-shot, human-reviewed **backfill** for recipes written before the link existed. A link chosen once by a human is deterministic forever; save-time fuzzy matching would re-guess on every save.

- **The cook's words stay.** `ingredients.name` remains free text and is what every surface renders; `food_id` is metadata no card ever shows. Forcing canonical names would fight the product's core claim (structure and intuitiveness of the recipe).
- **Provenance is a source key inside the existing nutrition jsonb** — `'auto'` when estimated; **absent means manual**, so every already-saved label and all ~1,320 sim labels read correctly with zero migration. `null` column stays the one representation of "no info", so **None** and Manual-all-empty collapse to the same state on purpose.
- **Arithmetic exists once, in SQL.** `estimate_nutrition()` is the preview; `save_recipe` recomputes through the same internals whenever the incoming label says `source = 'auto'` — client-sent auto numbers are preview-only and never stored. No Dart mirror (the Gotcha 19 twin risk is not bought when nothing needs per-keystroke recompute).
- **The registry is committed data, not a runtime dependency.** `nutritionData/foods.json` (~400–600 curated foods; the corpus needs 237 names today) carries per-100 g values extracted *once* from the CSVs by `tool/fdc.dart`; `tool/nutrition.dart` generates `supabase/nutrition_foods.sql` from the JSON alone, so CI checks staleness without the bundle —
the recipeData pattern, split into extract + gen.
- **Honesty over coverage.** An unlinked ingredient, an unresolvable unit (`handful`, `pinch`, `to taste`), a null quantity, or an `is_optional` row contributes nothing and is listed as "not counted" in the editor; the label carries a generic `Estimated from ingredients` footnote (per-count footnote considered and dropped — it would grow the pinned key set for a line the editor already shows better).

29a — Food registry & pipeline — DONE (2026-08-25)

- ☒ `nutritionData/` — `foods.json` (slug, display name, `fdc_id`, aliases, per-100 g values, `grams_per_ml`, named portions, `is_added_sugar`), `units.json` (canonical spellings + class: mass / volume / count / unresolvable), `README.md` authoring workflow. **78 foods, 277 aliases, 174 portions committed.** Authored fields win over the machine-written `extracted` block, per unit key / per value
- ☒ `tool/fdc.dart` — **extract:** reads the CSV bundle (path by argument, never committed), writes values into `foods.json`; energy fallback 1008 → 2047 → 2048, sugars id 2000, SR-Legacy `modifier` portion parsing (parenthetical strip → first comma segment → spelling/size/display-name-token match), derived `grams_per_ml` from the largest volume portion; run against the 2026-04-30 bundle
- ☒ `tool/nutrition.dart` — **gen:** `foods.json + units.json → supabase/nutrition_foods.sql`; melos `nutrition:validate / nutrition:gen / nutrition:check` (CI), pure file ops like `recipes:*`
- ☒ Schema in `0001_init.sql` (pre-release, folded in): `food` (11 per-100 g numeric columns — FDC's EAV flattened, named exactly as the label's jsonb keys), `food_alias`, `food_portion`, `food_unit` (`units.json`'s SQL mirror, one row per spelling — added so 29c's estimator reads a table instead of restating conversions), `pg_trgm`, `search_foods(q, lim)` RPC (exact > prefix > trigram, total order, capped 25); RLS select-only for `authenticated`, zero write policies, write grants revoked (Gotcha 4), RPC revoked from `anon` (Gotcha 3). No `search_tsv` generated column — aliases live in another table, so the typeahead is prefix + trigram over both

- ☒ Load order: `db:nutrition` melos script; `db:reset` is drop → create → **nutrition** → seed → recipes → sim; `config.toml sql_paths` gains the file **before** `seed_recipes.sql` (29b's FK makes the order load-bearing); `drop.sql` learns all four tables + the RPC; `db:clean` spares the registry (it never listed it); `database.yml` applies the registry on the fresh, re-apply, and upgrade paths and smokes `search_foods`
- ☒ `rls_matrix.sql`: new section E (E1–E9, 79 → **88 checks**) — registry readable signed-in, all four tables' writes fail **42501**, `anon` reads empty, `search_foods` finds a fixture signed-in and fails **42501** as `anon`. Proven non-vacuous the BL-7 way — E2 needs **both** locks (grant + policy) removed to go red, which is the defense-in-depth working
- ☒ Seed the vocabulary from the corpus: all 104 distinct recipeData names mapped — 137/147 ingredient rows linkable; the 7 unlinkable names (`mirin`, `orange liqueur`, `tikka spice blend`, `bamboo skewers`, the two salt-and-pepper composites, one serving suggestion) are documented gaps in `nutritionData/README.md`. `simData`'s ~111 unmatched names are 29d's optional curation, per the plan
- Found and fixed during verification: **B074** — the documented B033 psql pattern (`Get-Content -Raw | docker exec`) mojibakes every multibyte character; byte-faithful `cmd /c` redirection is the corrected form, local DB rebuilt clean. Hosted probe is an open owner action (see the tracker)

29b — Ingredient links at input — DONE (2026-08-25)

- ☒ `ingredients.food_id` — nullable `references food(id) on delete set null`, declared via `alter table ... if not exists after` the registry tables (fresh-apply order: `ingredients` is created before `food` exists), plus `ingredients_food_idx`. Every restatement site in the same change: `fork_recipe`, `save_recipe` (payload key `food_id`, no signature change), `seed_recipe_v2` via `tool/recipes.dart` (same — the key rides the existing jsonb argument), the sim's insert (null pass-through until 29d's optional curation), `Ingredient.foodId`, `EditIngredient.foodId` (B035 — the round-trip test covers it), both `schema.jsons` + `tool/recipe_format.dart` (optional `food` key; the slug must exist in `foods.json`, loaded by the new `loadFoodSlugs()`). Read side free — `kRecipeDetailSelect` embeds `ingredients(*)`, `recipe_snapshot` is `to_jsonb`
- ☒ Typeahead in `ingredients_editor.dart`: `RawAutocomplete` over the name field against `search_foods` (250 ms debounce, ≥ 2 chars, failures collapse to "no suggestions" — a hint surface, never a gate); picking sets name + link, typing past it leaves free text and **renaming keeps the link** (surviving a rename is the point of the FK); a linked row shows an `InputChip` on its own line (clearable, label ellipses); envelope re-run at 320/360/600 × 2.0 with the chip present (Gotcha 26)
- ☒ `FoodRepository` (abstract + `SupabaseFoodRepository`, wired in core providers): `search()` + `displayNames()` (labels the chips of a loaded recipe; empty input makes no request), with `fake_supabase` request-assertion tests; `FoodHit` model
- ☒ `recipeData/recipes/*.json`: `food` slugs on all 137 linkable ingredient rows (the 7 documented unlinkable names stay free text); `seed_recipes.sql` + `1_sim_dishes.sql` regenerated and committed
- ☒ `rls_matrix.sql` B22b (88 → **89 checks**): the owner's `save_recipe` stores the ingredient link, read back through RLS

- ☒ `estimate_nutrition(p_ingredient_groups jsonb, p_servings int) → jsonb` — pure over its arguments plus the registry (it never reads `recipes/ingredients`, so an **unsaved** draft previews through the same function the save path recomputes with); grams ladder: $\text{mass} \times \text{food_unit.factor} / \text{volume} \times \text{factor} \times \text{grams_per_ml} / \text{count}$ via `food_portion` ('' = bare count) / else skip; skips optional + unlinked + nameless rows and any `quantity` that is null **or** ≤ 0 (**B076** — the column has no positive check, and a negative row would subtract from the label rather than be absent from it); added sugars = authored `added_sugars_g`, else Σ total sugars of `is_added_sugar` foods; $\div \text{greatest}(\text{servings}, 1)$; kcal/mg rounded whole, grams to one decimal; returns `{label, counted, total, unmatched}` with the label **null** when nothing counted and `source: 'auto'` stamped when it did. `authenticated` only
- ☒ `match_foods(names text[]) → jsonb` — batched top-3 `search_foods` candidates per trimmed name, `[]` (not an absent key) when nothing matches, capped at 100 names
- ☒ `save_recipe`: when `nutrition->>'source' = 'auto'`, discard client numbers and recompute from the incoming trees against the **effective** servings (payload's, else the row's, read `for update`); manual/null pass through untouched. Signature unchanged — no 42725 exposure. Found **B075** doing it: the recomputed label needs its own `nullif(..., 'null'::jsonb)` or the nothing-counted case aborts the save with **23514**
- ☒ `RecipeNutrition.source` (`String?`, `includeIfNull: false`, `isEmpty` ignores it, `isEstimated` getter); `_nutritionKeys` learns `source` (validator: `'auto'` is its only value, and a `{source}`-only object is rejected like an empty one); label never renders it
- ☒ `NutritionEstimate` model + `FoodRepository.estimate()` / `matchFoods()`; the RPC trees come from a new shared `content_payload.dart` that `save_recipe`'s caller uses too, so the preview can never estimate a different tree than the one that saves
- ☒ Editor: **Automatic / Manual / None** (`ChoiceChips` in a `Wrap`, the Gotcha 21 shape — not `SegmentedButton`); Auto pane = counted-of-total header, preview label, not-counted list with a per-row reason and tappable `match_foods` chips that link the row; zero counted → inline warning, saves **null** rather than an empty lie; Auto → Manual seeds the fields with the **fresh** computed values (not the stale stored ones); Manual → Auto confirms before discarding typed numbers. The estimate refreshes on the three discrete events **and** on a 500 ms debounce from the ingredient list and the servings field (**B077** — the pane's own copy sends the cook to the list below, and servings is the divisor)
- ☒ `NutritionFactsLabel`: optional `Estimated from ingredients – not a measured analysis`. footnote; `nutrition_tab.dart` passes `nutrition.isEstimated`
- ☒ `supabase/tests/nutrition_estimate.sql` — fixture foods/units/trees → exact expected labels (whole ladder, nothing-counted, null/zero servings, rounding + the authored added-sugars override, empty tree, `match_foods`), rolled back; wired into `database.yml` (this SQL's only coverage, the `3_sim_verify.sql` rationale); `rls_matrix.sql` gains **B22c** (a save claiming `auto` with `calories: 9999` stores the recomputed 100) and **B22d** (auto with nothing counted stores SQL NULL) — 89 → **91 checks**

- ☒ Replaced the two all-10 placeholder labels: `estimate_nutrition` run over all 14 linked authored recipes against the real registry, and the output committed — **12 source: 'auto'** (coverage 4/6 to 17/19), **fresh-guacamole manual** (two rows are "to taste", so auto counts 4 of 6 — the honest reason a cook types the numbers), and **classic-margarita null**. All three editor modes therefore demo on seed alone (the Seed-data fit gate); `seed_recipes.sql` regenerated and committed
- ☒ `recompute_auto_nutrition()` in `0001_init.sql` + `select` on every apply, the `recompute_all_chef_stats()` pattern. Estimates from `recipe_snapshot(id) -> 'ingredient_groups'` (the same tree `save_recipe` persisted, never a hand-rolled one), touches only `source = 'auto', is distinct from`-guarded so an apply that changes nothing does not bump `updated_at` through `recipes_touch`, and **returns early on an empty registry** — `db:reset` and CI's upgrade path both apply `0001 before nutrition_foods.sql`, so without that guard the on-apply call blanks every stored label. Invoker-rights, `execute` revoked (Gotcha 3); listed in `drop.sql` (B024 discipline)
- ☒ `rls_matrix.sql` **E10** (91 -> **92 checks**): a signed-in user calling `recompute_auto_nutrition()` must fail **42501**. Double-locked like E2 — the function's own revoke *and* `recipe_snapshot`'s — so the non-vacuity ritual had to hand both grants back before it went red
- ☒ `supabase/tests/nutrition_fixtures.sql` (`melos run db:nutrition:verify`, wired into `database.yml` after the fresh apply) — the drift gate the pipeline was missing: `recipes:check` proves the SQL matches `recipeData/`, nothing proved the numbers inside it still match `nutritionData/`. Asserts all three states exist, every committed auto label equals its recompute, and then breaks three things inside a rolled-back transaction: a corrupted auto label must come back, a manual label must not move, a deleted registry must blank nothing. **All four proven non-vacuous** by reverting the code each covers
- ☒ Sim untouched, as planned: its invented labels carry no `source`, so they read as manual and the backfill's predicate never selects them (confirmed against a generated population — 12 auto of 87 labelled). Linking `simData` ingredients stays optional follow-up curation
- ☒ Docs: SDS §3.1 (the refresh path, its four load-bearing properties) + §11.1 (the three-way fixture split), CLAUDE.md (nutrition paragraph, snapshot-not-live-view rule, commands, repo layout, the Bash-side `docker cp` form of B033/B074), README (`fdc:extract` bundle path, the two new `db:nutrition:*` scripts), `recipeData/{README,schema}.json` (`source` documented, the regenerate-on-change rule), BL-5 register

Verification plan

- ☒ `melos run analyze` (SUCCESS) / `test --no-select` (218 app + 122 core + 115 design_system, all passed) / `recipes:check · sim:check · nutrition:check` (SUCCESS)
- ☒ **29d: melos run db:rls — 92 passed, 0 failed** (was 91), E10 proven non-vacuous by granting both `recompute_auto_nutrition()` and `recipe_snapshot` back to `authenticated`. `db:nutrition:verify` green on the clean, seeded and upgraded databases, and each of its four assertions proven non-vacuous by reverting the code it covers — no-op backfill

→ \$3 red, predicate widened past `source = 'auto'` → \$4 red, empty-registry guard removed
→ \$5 red, a registry calorie value moved by 5 → \$2 red. Sim regenerated and
`3_sim_verify.sql` re-run (ALL CHECKS PASSED) to confirm the backfill leaves invented labels alone

- ☒ Local stack: the Gotcha 6 upgrade path (today's baseline over the running database) **and** the truly-clean B045 path (`drop.sql` → 0001 → nutrition → seed → seed_recipes), both green — `estimate_nutrition` reading tables a later file loads is exactly that class
- ☒ `melos run db:rls` — **91 passed, 0 failed** (was 89). Non-vacuity the BL-7 way: with the recompute branch replaced by `if false then`, B22c and B22d both go red and the fabricated `calories: 9999` is what lands in the column
- ☒ `supabase/tests/nutrition_estimate.sql` — all assertions passed, on both the upgraded and the freshly rebuilt database
- ☒ Editor envelope with the mode chips + Auto pane (preview label, not-counted list, suggestion chips) at 320 / 360 / 600 × 2.0×; detail suite asserts the footnote on an estimated label and its absence on a manual one
- ☒ Estimator sanity against the **real** curated registry (not fixtures): all 14 authored recipes produce plausible per-serving labels at high coverage (Spring Vegetable Tart 17/19, Tuna Fishcakes 15/16, Pancakes 8/8, Cookies 9/9), and the unmatched names are exactly the documented vocabulary gaps (`tikka spice blend`, `mirin`, `salt and pepper`, `bamboo skewers`). The eight recipes that estimate 0/N are all `seed.sql demo` recipes, which predate 29b and carry no `food_id` — a fixture fact, not an estimator fault
- ☐ Screenshots (B028) — **not run**. The Auto pane lives behind the auth redirect, and Flutter web exposes no DOM nodes, so a headless driver cannot sign in or type into the editor to reach it (CLAUDE.md's own B028 note). Covered instead by the widget envelope above; the label footnote adds no colour literal (theme `onSurfaceVariant` + italic), so the dark-mode risk it would have checked is not present

Deferred (out of scope for 29)

- ☐ Cooking yield / moisture loss — raw-ingredient sums, permanently disclosed as an estimate; not fixable with this dataset
- ☐ Micronutrients + per-100 g display (carried from Phase 28)
- ☐ Vocabulary mining loop (surfacing unlinked ingredient names as curation candidates)
- ☐ `simData` ingredient links (optional curation, above)

Phase 30 — Public chef page (`/chef/:id`) — DONE (2026-08-25)

Design, build order, and the traps: [EXECUTION-PLAN.md Phase 30](#).

Planned and built 2026-08-25. Three things the plan did not anticipate, each recorded where it belongs: handing the paged notifier its chef through a `StateProvider` written in `initState` throws `Tried to modify a provider while the widget tree was building`, so the argument arrives as

a **scoped `ProviderScope` override** instead (and the notifier must be overridden alongside its input, or it is created in the root container and reads the empty default); starting the profile

and standing reads together and awaiting them **sequentially** left the second future with no handler attached, so a fast failure surfaced as an unhandled async error *beside* the one the provider reported — `Future.wait` subscribes to both up front; and the `dense_rank` trap the plan predicted is real and now pinned by `rls_matrix.sql` **F3**, proven by breaking the function and watching it report `rank 1, 0 matching board row`.

Deferred since Phase 22 and pulled forward now because it is **Phase 25's first prerequisite**: a restaurant page is the same shape (identity header + recipe grid) and a member-chef row needs a destination. Today a chef tap opens a modal, there is no route listing one chef's public recipes, and a chef cannot be linked or shared.

Owner's decision (2026-08-25): the page replaces the dialog. `chef_detail_sheet.dart` is deleted; every chef tap navigates. The trade was argued both ways and is recorded in the execution plan — what is lost is peek-without-leaving-the-board, what is gained is a deep-linkable destination. Nothing is lost from the content: the page composes the same OPT-A8 panels.

Schema (`0001_init.sql`, idempotent)

- ☒ `chef_standing(p_chef uuid)` — the leaderboard's row shape for **one** chef, `stable`, invoker-rights, `anon`-callable. A **new name**, not a `chefs_leaderboard` overload (B024 / Gotcha 5), and the `dense_rank()` is computed over the full population then filtered — rank the filtered set instead and every chef comes back rank 1. Verified against the board for all 13 local chefs, ties included
- ☒ **Index — measured, and deliberately not added.** `recipes_owner_idx (owner_id)` already exists (`0001_init.sql:142`) and serves the equality; only the `created_at desc, id` sort is unindexed, over at most a few dozen rows per chef. Adding a partial index on that evidence would be exactly the blind optimization the plan was corrected to avoid — revisit with an `explain analyze` when a chef owns enough recipes for the sort to show up
- ☒ `drop.sql` entry + the in-file B024 drop block

core

- ☒ `ChefRepository.standing(chefId) → ChefStanding?`; **null means "profile exists but is not a chef"** — `chefs_leaderboard` excludes `public_recipe_count = 0`, so the private-only seed chef `d6` has no rank and the page renders that rather than 404. Decodes from the RPC array rather than `.single()`, which would turn the legitimate empty answer into `PGRST116`
- ☒ `RecipeRepository.listByChef` beside `listMine` (same shape, one filter apart), total order ending `created_at desc, id` (Gotcha 24). The explicit `visibility = 'public'` filter is **load-bearing**: RLS hands a signed-in owner their own private rows, so without it a chef's public page shows them what nobody else can see
- ☒ Signed-out safe by construction — `listByChef` takes the chef's id and never touches `_uid` (Gotcha 9)

app

- ☒ `/chef/:id` on the **root** navigator (like recipe detail), signed-out safe, **not** in `needsAuth`; `Routes.chef(id)` **and** `Routes.chefPattern`, both, per OPT-A8. Singular

/chef/ so it cannot collide with the plural /chefs

- ☒ **No nav destination** — Gotcha 18: a fifth costs the web pill its labels
- ☒ `features/chefs/chef_page.dart` — identity header, `ChefScorePanel`, then `RecipeAsyncSliverGrid` under a `PUBLIC RECIPES (n)` rule; one `CustomScrollView` since the page owns its scroll, and `showChef: false` because naming the chef on every one of their own cards is noise
- ☒ `chef_identity_header.dart` — the sheet's private `_Header`, moved and **re-based on Profile with an optional ChefStanding**. That is the phase in miniature: the board always had a standing to hand its card, a URL has neither, so name/avatar/tier now come from the profile and only the rank line depends on the standing
- ☒ Three `showChefDetail` call sites in `chefs_screen.dart` → `context.push`; `chef_detail_sheet.dart` and `chef_recipes_panel.dart` **deleted**
- [~] `ChefBadge.onTap` wired on **recipe detail's badge, both layouts** — the badge under the title now opens the chef. **Not** wired on the `RecipeCard` cover overlay: that needs a new `RecipeCard` parameter threaded through `RecipeGrid/SliverRecipeGrid` to six surfaces, and a second tap target inside an already-tappable card is an interaction decision, not a wiring one. Phase 18's `ChefBadge.onTap` **coverage** item is closed by the 2026-08-25 sweep (the branch is tested at the widget level and at both detail call sites); what stays open is the second *site*, listed under Deferred below

Seed-data fit — **outcome 1: existing data covers it**

- ☒ Checked before planning. Kitchen's 14 public recipes give a genuinely paged grid; d1–d7 give a chef per tier, Dara at exactly 100, and the Chen Wei / Greta rank-4 tie; **d6 private-only** gives the not-a-chef state on seed alone; `db:sim` at `medium` gives ~172 chefs for a real rank denominator. **No fixture change needed**. What seed cannot show: no profile carries an `avatar_url`, so the header's photo path ships unexercised (a standing BL-5 limit)

Verified

- ☒ `melos run analyze` — **No issues found!** ×3 · `melos run format` (5 files reflowed) · `melos run test --no-select` — core **122**, design_system **115**, app **235** (was 218)
- ☒ Local stack: the Gotcha 6 **upgrade path** — today's baseline applied over the running database, exit 0. `chef_standing` cross-checked against `chefs_leaderboard` for **all 13** local chefs: every rank matches, the tie at rank 4 is preserved, Dara Nilsson lands at 7 (not 1), `d6` returns 0 rows, an unknown uuid returns 0 rows, and `proac1` matches its two peer RPCs exactly. The *fresh* path is left to CI's `database.yml`, which runs it on every push — the new function references only `profiles` and the `chef_tier` enum, both created earlier in the same file, so the B045 shape does not apply
- ☒ `melos run db:rls` — **98 passed, 0 failed** (was 92). New section **F**, which also closes the three chef checks ROADMAP Phase 18 had listed as unasserted since 2026-08-19: `dense_rank` ties (F6), the `public_recipe_count = 0` exclusion from both directions (F4/F5), and `anon` calling `chefs_leaderboard` at all (F1) — every prior run proved that one as `postgres`, which bypasses grants and therefore proved nothing
- ☒ **F3 proven non-vacuous the BL-7 way**: `chef_standing` rewritten with the filter *inside* the window, run, and F3 went red reporting `rank 1, 0 matching board row`. A count-only F6

stayed **green** under that same break, which is why F6 now compares against the board's rank rather than merely counting distinct ranks

- ☒ Envelope at 320 / 390 / 600 / 1000 / 1440 × {1.0, 2.0} with a 38-character chef name, a six-figure score and a bio — no overflow at any of the ten combinations
- ☒ `chefs_screen_test.dart`'s `ChefDetailView` assertions **rewritten** as navigation assertions, not deleted, plus one new check that the *tapped* row decides the chef (a tied row, so an implementation keying off rank instead of id lands wrong). The dialog's content assertions moved to the new `chef_page_test.dart` (17 tests)
- ☒ **Screenshots (B028) — run 2026-08-25, no defects.** The page is signed-out safe and URL-reachable, so unlike Phase 29's editor pane a headless driver *can* reach it: release build + `npx serve` + Playwright/Chrome, navigating by hash URL. **Three states**, against the local stack with the sim loaded — and note which width and theme each actually got, so the gaps are visible rather than implied:
the **ranked** chef (Kitchen — rank 2, head chef, 14 recipes) at 1440×1000 and 390×1600, light *and* dark; the **not-ranked** chef (d6 Farid Haddad — private-only) at 1440, light and dark, showing `Not ranked yet · 0 public recipes · joined Aug 2026`, the explanatory line in place of the score panel, and the grid's empty state; and the **paged** case (a sim chef with 23 public recipes) at 1440, **light only**, showing `Load more` after exactly 20 — `kRecipePageSize`, so the page boundary lands where OPT-P9 says it does. Also confirmed visually: `showChef: false` (no badge on a single card of the chef's own grid), the tier tint changing the header band per chef, and a Cyrillic display name rendering with correct initials. Dark mode is legible everywhere it was shot — no B055-class colour defect, which is the one thing a layout test cannot see. Procedure per BL-6: `themeMode` pinned in `main.dart`, rebuilt, served on a **new port**, then reverted and `git diff` confirmed empty.
The header carries no rank denominator — it reads `Rank 2 · 14 public recipes`, not the dialog's old `Rank 2 of 148`, since `chef_standing` returns a rank and not a population size

Deferred / follow-ups

- ☐ `chef_top_recipes` and `ChefRepository.topRecipes` **now have no caller**. They backed the dialog's "Top recipes" list; the page shows the chef's whole catalogue instead, so a top-3 section above it would print the same recipes twice. Both were **kept, not deleted**: the RPC is granted, tested and exactly the query shape Phase 25's signature dishes will want, and deleting the Dart method would delete its repository test with it. Worth a decision rather than a silent life as dead code — prune, or use it
- ☐ `ChefBadge.onTap` on the `RecipeCard` cover overlay (see the app section above) — needs a `RecipeCard` parameter threaded to six surfaces plus a call on nested tap targets
- ☐ The page has no cover/banner art. The identity header is a tinted band, because no seeded profile carries an `avatar_url` and there is no chef cover column at all
- ☐ Nothing links to a chef from Discover — the card overlay above is that link

Phase OPT — Optimization & hardening backlog (rolling)

Findings from the 2026-08-20 design/architecture audit (Dart + SQL), plus the open items prior phases deferred, consolidated in one place. Detail — mechanism, fix shape, acceptance — in

EXECUTION-PLAN.md Phase OPT.

This phase is **rolling**: items are picked by priority band, not executed top-to-bottom in one sitting. New audit findings land here; **Bxxx** tags mean the mechanism is in **BUG-TRACKER.md**.

Status 2026-08-22: 26 of 29 items done; the phase is closed for execution. The three that remain are not scheduled work — one is an owner action on production, one is accepted debt, one is a deliberate deferral — so they moved to **Backlog** as **BL-1**, **BL-2**, **BL-4** and are marked **[→]** below. Nothing in Phase OPT is waiting on a decision except those.

OPT-S — Integrity & correctness (do these first)

- ☒ **OPT-S1 (B050, high)**: column-level **INSERT/UPDATE** grants on **recipes** / **profiles** — an owner could forge **like_count** / **chef_score** over PostgREST and the leaderboard believed it. Also made **current_version_id** server-owned for real (**recipe_versions_set_current** trigger) so it could leave the grant list. Verified on the local stack, both the upgrade path and a clean reset
- ☒ **OPT-S1a (B053, high)**: **recipes_select** called **can_read_recipe(id)**, a **stable** function that cannot see the row an **INSERT ... RETURNING** is inserting — so **creating a recipe failed outright**. Policy inlined against the row's own columns. Found by OPT-S1's acceptance matrix; pre-existing. Also 2.4× faster on a Discover scan (15.7 ms → 6.5 ms at sim **medium**)
- ☒ **OPT-S2**: added **.select()** + empty-result check to the **recipes update()** / **delete()** calls in **recipe_repository.dart**, plus **unshare()** (same class, same one-line fix); they throw **WriteDeniedException** instead of reporting a silent no-op as success (Gotcha 2)
- ☒ **OPT-S3 (B051)**: recipe-detail like/save — guards signed-out (routes to **/auth**), reads **myLiked/mySaved**, both are toggles, failures surface. New **recipe_detail_test.dart** (5 tests) is the first slice of OPT-T3's recipe-detail suite
- ☒ **OPT-S4 (B052)**: recipe-editor **_load()** failure path — catches, renders **ErrorView** with retry instead of the form, and **_canSave** blocks Save until a load has succeeded (4 tests)
- ☒ **OPT-S5**: share dialog no longer offers a working-looking "Can edit" — the segment is **enabled: false** behind **notYetTooltip**, which moved to **apps/app/lib/widgets/** so the dialog isn't importing out of **features/chefs/** (pre-empts an OPT-A3 item)
- ☒ **OPT-S6**: **fork_recipe** — explicit **auth.uid() is null** guard **and** **EXECUTE** revoked from **public/anon**, granted back to **authenticated**; both locks verified on the local stack
- ☒ **OPT-S7 (B034)**: **SUPABASE_DB_URL** moved out of the dart-define files into a dot-sourced, git-ignored **db-url.local.ps1** (template committed as **db-url.example.ps1**). Per-shell and per-project rather than a global user env var, so it cannot leak into another repo's **db:*** run. History checked: the credential was never committed, so no rotation needed
- **[→] OPT-S8 (B018)**: rotate the pre-fix seed accounts on the **hosted** project. **supabase/scripts/rotate_seed_passwords.sql** is written and verified on the local stack (16/16 rotated, recipes/ratings/profiles intact) — **running it against hosted is the owner's action**, deliberately not automated: it writes to **auth.users** on production.
→ **Backlog BL-1**

OPT-P — Performance & scalability (sim **medium** makes these measurable)

- **OPT-P1:** `recipes_search` recomputed `recipe_search_document()` twice per public recipe per search. Now a trigger-maintained `recipes.search_tsv` + GIN index: **539.6 ms** → **~1–2.5 ms (>200×)** at sim `medium`, identical result set (304 matches). Statement-level triggers with transition tables cover title/description, ingredients, group cascade-deletes, tagging and tag renames. `kRecipeSelect` moved off `*` so the ~450-byte tsvector doesn't ship (~13 KB per 30-card page)
- **OPT-P2:** `recipes_trending` bounded to 30 days + partial index `recipes_public_created_idx (created_at desc) where visibility='public'`. At sim `medium`: trending **23.4 ms** → **3.9 ms** (6.0×, 1,344 → 351 rows scored); Discover **Recent** **1.1 ms** → **0.23 ms** and now a pure index scan with the sort eliminated
- **OPT-P3:** `getById` was 2+G+S round trips (one per group) — now **one** nested PostgREST embed with foreign-table ordering, B022's explicit ascending kept at all four levels. **4.69 requests** → **1 on average**, worst case 8 → 1. Needed `@JsonKey(name:)` on `Recipe.ingredientGroups/stepGroups`, without which content silently decoded as empty
- **OPT-P4:** one editor save cost 2 full `getById` cascades (`_appendVersion` + the return). Now **1**: the post-save read feeds both the snapshot and the return value, and `_appendVersion` returns the new version id so the pointer is carried over rather than re-read. Snapshot is the post-save read on purpose — the caller's draft has stale ids
- **OPT-P5:** `chefs_leaderboard` re-aggregated all public recipes per page for numbers `recompute_chef_stats` already had — `profiles.total_likes/saves/views` now persist them and the board reads them straight off the row: **3.5 ms** → **0.5 ms** warm at sim `medium` (52 ms cold), zero aggregation. New `recompute_all_chef_stats()` replaces the same UPDATE restated in three files (backfill, sim generate, sim teardown) — Gotcha 19 in one place. Partial `profiles_leaderboard_idx` on the board's exact ordering supersedes `profiles_chef_score_idx`; new sim assertion A6b fails if a total ever drifts from the recipes behind it
- **OPT-P6:** added `(recipe_id, created_at desc)` to `recipe_likes` / `recipe_saves` — both PKs lead with `user_id`, so every recipe-leading read was a seq scan. "Who liked recipe X, newest first" goes seq-scan-over-6,483-rows + sort → **pure index scan, sort eliminated**; write paths still PK-served. Unblocks Phase 23's windowed rails
- **OPT-P7:** `logView` fired on every `recipeProvider` re-resolution, so each like/save/rate appended another `recipe_views` row. Moved to its own `recipeViewLoggerProvider` (`autoDispose` = one per visit); 3 tests pin it
- **OPT-P8:** Discover search fired per keystroke — 300 ms debounce in `searchResultsProvider` (`kSearchDebounce`), superseded queries cancelled on dispose so they never reach the network; 4 tests pin it
- **OPT-P9:** pagination for all six browsing surfaces — Discover's four lists and both My Recipes tabs. `p_offset` on the three Discover RPCs, `.range()` on the table reads, and a **total** order everywhere (`... , created_at desc, id`) so a page boundary cannot duplicate or skip a row. Client state is one `PagedRecipesNotifier` in `core`; every surface renders through the new `RecipeAsyncGrid`, which also absorbed the duplicated Loading/Error/Empty/grid ladder (OPT-A7's grid item, done early). **Load more button, not infinite scroll** — owner's call. 7 tests in `paging_test.dart`

- ☒ **OPT-P10:** `/chefs` hero issued 6 count queries → **1** `chefs_tier_counts()` RPC; `chefCount()` deleted because the total is the sum of the tiers, and `chefCountProvider` now derives it (4 call sites share the one request). `chefDetailProvider` starts both requests before awaiting either, keeping the non-fatal top-recipes fallback
- [→] **OPT-P11 (accepted, revisit before growth):** per-engagement-row `recompute_chef_stats` is a full aggregate per like/first-view — fine now (SDS §10.3), the hot-path cost later.
→ [Backlog BL-2](#)

OPT-A — Architecture & code quality

- ☒ **OPT-A1:** `save_recipe(...)` — one transactional RPC for create/update + content replace + version append. Closes Gotcha 11's data-loss window, computes `version_number` under the row lock (race gone), and collapses a save from ~10 requests to 2 (the call + one read). `recipe_snapshot()` builds the version snapshot server-side and also fixed `fork_recipe`, whose first version stored a literal `'{}'`. Verified end to end on the local stack with a throwaway harness (create, edit with reorder, non-owner refused, signed-out refused), then deleted per Gotcha 15
- ☒ **OPT-A2:** deleted the retired `features/home/home_screen.dart` (185 dead lines) and its directory; nothing imported it and `widget_test.dart` already pins the `/` redirect
- ☒ **OPT-A3:** `share_dialog.dart` → `apps/app/lib/widgets/` (it was in `my_recipes`, which never opened it — `recipe_detail` did); sign-out is a direct `authRepositoryProvider.signOut()` at both call sites instead of a cross-feature import of `AuthController`, whose now-unused `signOut()` went with it
- ☒ **OPT-A4:** `friendlyError()` in core maps `PostgrestException` codes (42501, 23505, 23503, 23514, PGRST116/301/202), `AuthException` (passed through — GoTrue writes those for users), `StorageException`, `WriteDeniedException`, the signed-out `StateError`, and network faults to one actionable sentence; it logs the raw error itself, so no call site does. All 14 surfaces converted (Discover/My grid, chefs board + sheet, recipe detail incl. fork/rate, editor load + save, profile, auth, share dialog). 8 tests
- ☒ **OPT-A5:** `findByEmailOrName` → `searchByName`, a `%query%` match returning up to 8 profiles ranked exact → prefix → contains (LIKE wildcards in the query escaped). `ShareDialog` now debounces the lookup, lists the matches with avatar + tier, excludes you, auto-selects a lone match, and keeps **Share disabled until one is chosen** — with three "Amara Baptiste"s in the sim data, "share with Amara" was picking one at random and reporting success. 6 tests (also closes OPT-T3's share-dialog item)
- ☒ **OPT-A6:** five schema nits, one local-stack pass. Avatars bucket gained its delete policy (parity with recipe-images — a replaced avatar was unreachable-but-public forever); `chefs_leaderboard` gained its B024 drop-guard; redundant `recipe_versions_recipe_idx` dropped (the `(recipe_id, version_number)` unique already leads with `recipe_id` — verified `versions()` still index-scans); the two deferred FKs are added only when absent, so a re-apply no longer revalidates every `recipes` row; `tags` keeps free creation but gained **delete-when-orphaned** and stays un-updatable — proven both ways on the local stack
- ☒ **OPT-A7:** three dedupes. `StorageService`'s two upload bodies differed by bucket name alone → one `_upload(bucket, ...)`; the grid scaffold landed with OPT-P9 (`RecipeAsyncGrid`); the two hand-rolled date formatters became `monthYear` / `isoDate` in `formatting.dart` (2 tests — the month table is 0-indexed and the ISO one pads two fields)

- ☒ **OPT-A8:** the three oversized screens split along their real seams — editor **880** → **418** (`cover_picker`, `ingredients_editor`, `steps_editor`), detail **629** → **311** (`rating_section`, `recipe_content_views`, `detail_chips`), chef sheet **597** → **231** (`chef_score_panel`, `chef_recipes_panel`, `chef_detail_common`). Pure moves: no widget changed, only its address. Route literals gone from the route table **and** from three feature call sites the plan had not spotted — `Routes.recipePattern` / `editRecipePattern` are the new match-side constants
- ☒ **OPT-A9:** `supabase/migrations/` is a numbered sequence and `0001_init.sql` is the **frozen baseline**; the next schema change is `0002_*.sql`. `db:create` applies the whole directory in filename order, and the hosted procedure is now "apply only the new file" — so shipping a one-liner stops re-running the baseline's two whole-table backfills. The baseline's content was deliberately **not** chopped into pieces: that would trade a real property (one re-applicable file every doc and script depends on) for tidiness.
Partially reversed 2026-08-23 (owner): while the project is pre-release the baseline is editable again, so Phase 26's shelves live in 0001 and `0002` was deleted. The machinery (`db:create` over the whole directory, "apply only the new file" for hosted) is unchanged and the freeze resumes the day the schema ships anywhere real.
`supabase/migrations/README.md` carries the rules, including the B024 drop discipline

OPT-T — Tests, tooling & process (existing debt, consolidated)

- ☒ **OPT-T1:** `.github/workflows/database.yml` — a second job that starts the Supabase stack inside the runner and walks **three** paths: fresh apply → seed → recipes → sim tiny → the 46 assertions; a full re-apply (the only thing that checks the idempotency every file claims); and the **upgrade path** (Gotcha 6) — the previous revision of the baseline applied first, today's layered on top, which is the path B024 shipped through. No `SUPABASE_DB_URL` secret, by design. Verified by running the same sequence locally, which also turned up **B054** (`db:reset` leaves a stale sim registry)
- ☒ **OPT-T2:** repository tests exist — 14 of them, without mocking `SupabaseClient` at all. `SupabaseClient` takes an `httpClient`, so `test/support/fake_supabase.dart` records the request and replies from a queue; `signInAs` uses `recoverSession` to sign in offline. Pins the contracts that have broken here: the `kRecipeSelect` FK hint and column list, B022's four ascending embed orders, OPT-P3's one request per open, OPT-P9's `limit/offset` and total ordering, OPT-A1's single `save_recipe` body (writable columns only), the `42501` → `WriteDeniedException` translation, and the signed-out paths that must not reach the network
- ☒ **OPT-T3:** the whole list is closed. Recipe-detail interactions landed with OPT-S3, the share dialog with OPT-A5, and this item added `snapRating` (4 tests, including the property that every input in range produces a value the SQL check constraint accepts) and the auth screen (6 tests through the **real router**: which door `?mode=signup` opens, validation before network, a failed sign-in showing the mapped message and staying put)
- ☒ **OPT-T4 (2 of 3):** `pubspec.lock` committed for all four packages (B009 closed) and the `sdk`: bound raised to `>=3.7.0` with a single whole-repo reformat, so `melos run format` no longer breaks `melos run analyze` (B027 closed — verified in that order).
freesed 3.x is deliberately not done: it is a breaking model-syntax migration whose only prize is unpinning Flutter, which nothing needs today, and the plan already called it "its own change set". Tracked below

-  **OPT-T5:** screenshot pass done over Discover (390 / 700 / 1400), `/chefs` v3 at 1400, and a recipe detail at 1000, through the B028 procedure (release build + static serve). Branded Chrome still will not install (needs Administrator), so the pass runs on Playwright's bundled **Chromium** via `npx playwright screenshot` — the MCP browser is the only thing that needs `chrome`. Found and fixed **B055** (tier chip unreadable on a card cover — the defect is colour only, so no layout test could see it) and **B056** (ungrouped like/save counters). `/chefs` v3 verified correct: hero, tier tiles, tie ranks, score explanations, `1 recipe` singular
-  **OPT-T6:** `tool/db.dart` runs `create / seed / recipes / drop / clean` under `psql -1`, one transaction **per file**, so a mid-file failure rolls back instead of leaving a partial schema. The sim files stay exempt — they manage their own transactions and toggle triggers. Verified: all three big files apply cleanly under `-1`, and a deliberate mid-file error leaves nothing behind

- [→] **OPT-T4c:** migrate to `freezed` 3.x (B005's permanent fix — unpins Flutter). Breaking model syntax across every `@freezed` class, a `build_runner/analyzer` bump, and a full codegen + verification pass. Do it when a newer Flutter is actually wanted, not before.
→ [Backlog BL-4](#)

Outstanding (environment-dependent)

Toolchain is set up and verified (bootstrap, codegen, analyze, test, `flutter build web --release`), and the app runs against the hosted Supabase project with seeded data. These remain — all now tracked as [Backlog BL-6](#):

- **Signed-in flows:** create / edit / version history **were exercised end-to-end on the local stack** during OPT-P4 (throwaway account + harness, both deleted afterwards) — create with every column, the trigger-set `current_version_id`, an edit that adds an ingredient, ordering preserved across the delete-and-reinsert, both version snapshots, the search document refreshing, a private recipe staying out of public search, and a signed-out update throwing rather than reporting success. That run is what found the stale-pointer bug in P4's first cut. **Still not exercised:** fork from the UI, and Storage image upload.
- **Mobile/emulator** manual pass (no Android SDK installed on the current machine).
- ~~Repository unit tests with a mocked Supabase client~~ — **done by OPT-T2**, and without a mock: a recording `http.BaseClient` under a real `SupabaseClient`.
- ~~Squash the single idempotent `0001_init.sql` into versioned migrations~~ — **superseded by OPT-A9:** `supabase/migrations/` is a numbered sequence and the baseline is frozen, so a schema change ships as `0002_*.sql` rather than a re-apply of 0001. Chopping the baseline itself was deliberately rejected (it would trade a re-appliable file every doc and script depends on for tidiness).

Backlog — deferred, not scheduled

Everything here is **known, decided, and not being worked on**. An item is in the backlog because it is an owner action, accepted debt, or a deferral with a stated trigger — not because it was

forgotten. Each one names the condition that would pull it back into a phase.

What is open elsewhere in this document, so the backlog is not mistaken for the whole picture: Phases 0–23 and 26–29 shipped their scope (29a–d all landed 2026-08-25 and reached the hosted project the same day), each carrying its own **Deferred** block; **Phase 24 is [~] in progress** — the generator and its 46 assertions are done, but the dish library is 25 of 120, `people.json` / `vocab.json` / `sim.rand_zipf` are unwritten (SDS §12 **is** written now, 2026-08-25, and it records a new open question: CI's `tiny` preset skips the whole of verify group G);

Phase 30 shipped 2026-08-25

(the public chef page, pulled forward because it was Phase 25's first prerequisite — that prerequisite is now met); **Phase 25 is designed-not-started** behind its two remaining prerequisites, B043's tier calibration and the SQL harness (itself now closed, see Phase 11); and Phase OPT is closed at 26 of 29 with the rest listed below. Two single-item stragglers sit in otherwise-complete phases: per-step image upload (Phase 9) and the mobile/emulator pass (Phase 12, which is BL-6). Phase 28's Deferred block resolved into Phase 29 (auto-calculate and the real label values); micronutrients stays with the phase rather than here, a feature deferral with no trigger condition.

BL-1 — OPT-S8 (B018) — rotate the hosted seed passwords (owner action)

Nine seeded production accounts still carry the pre-fix literal passwords that were committed in `seed.sql`. `supabase/scripts/rotate_seed_passwords.sql` is written and verified on the local stack (16/16 rotated; recipes, ratings and profiles intact). **Deliberately not automated** — it writes to `auth.users` on production, so no script or CI job may hold that credential.

Trigger: run it whenever you next have the hosted DB URL in a shell. Until then, treat those accounts as compromised.

Deferred once, knowingly (2026-08-25). The Phase 29 hosted rollout had the pooler URI in a shell — precisely the stated trigger — and the owner's call was to skip it as non-priority. Noted so this does not keep reading as "the trigger has never come up": it has, once, and was declined. The trigger stands unchanged for the next such session.

BL-2 — OPT-P11 — per-engagement-row `recompute_chef_stats` (accepted debt)

Every like and every first-view runs a full aggregate over the actor's public recipes (SDS §10.3). Correct, and measured fine at `sim medium`. **Trigger:** a real engagement rate where writes contend — revisit as an incremental delta or a debounced recompute before growth, not now.

BL-3 — B054 — `db:reset` leaves a stale `sim` registry (needs a decision, not a patch)

`drop.sql` never touches schema `sim` and spares `auth.users` by design, so a reset on a machine that has run the `sim` leaves 1,000 simulated accounts and a registry describing recipes that no longer exist; the next `db:sim` builds on the ghost and `3_sim_verify.sql` fails E9 or A7. Making `drop.sql` drop schema `sim` is **worse** — it strands the `auth.users` rows with no registry to delete them by. The right shape is `db:reset` running `9_sim_takedown.sql` first, which makes `db:reset` a `--yes`-gated destructive action that deletes `auth.users` rows.

Trigger: owner's call on that gate. Workaround today: `melos run db:sim:clean -- --yes` before a reset.

BL-4 — OPT-T4c — migrate to **frozen** 3.x

B005's permanent fix; unpins Flutter 3.44.8. Breaking model syntax across every **@frozen** class plus a **build_runner/analyzer** bump and a full codegen + verification pass.

Trigger: the first time a newer Flutter is actually wanted. Its own change set, never the tail of a batch.

BL-5 — seed & sim coverage register (read this when planning a feature)

Not a task — the standing list of what the fixtures **cannot** demonstrate, so a feature is never designed onto data that does not exist. See the "Seed-data fit" gate in [CLAUDE.md](#). Known limits today:

- **seed.sql authors** counters (**like_count** = 2500) with no **recipe_likes** rows behind them, so any dated, windowed, or "who did this" query reads empty against demo data alone. The sim writes the rows and derives the counters — that is what makes SDS §10.8-style queries testable.
- The sim's time anchor is **sim.epoch_end()**, **pinned**, not **now()** (B044) — a feature that keys off "recent" must be checked against that anchor, not the wall clock.
- Teardown is registry-driven (B054 above), so a fixture a feature adds outside **sim.actor** / **sim.recipe** will not be cleaned up.
- 25 of a planned 120 simulation dishes are authored (Phase 24), so directory/category coverage is thin in places — check **simData/README.md**'s coverage rules before assuming a category populates.
- The 14 authored recipes top out at **85 minutes**, contain **no hard** difficulty and carry **no forks**, so a seed-only database cannot show anything ranked on long cooks or on lineage — Phase 26's shelves **02** and **03** are empty there by construction and say so on the page. Both need the sim.
- Fork **depth** comes from **sim.fork_bias** (Phase 26). Uniform source selection spreads forks one per recipe, which looks like data and ranks like nothing; anything ordered by fork count needs the weighted draw and the ≥ 3 assertion in **3_sim_verify.sql** §G.
- **Nutrition labels: recipeData is now real, the sim is still invented** (Phase 29d). All 14 authored recipes carry a genuine label — 12 estimated from their own ingredient links (**source: 'auto'**), **fresh-guacamole** manual, **classic-margarita** null — so **all three editor modes demo on seed alone**, which was the gap the all-10 placeholders left. What seed still cannot give you is **scale**: anything needing hundreds of varied labels needs **db:sim**, where **sim.nutrition_for** draws one per recipe from a per-category profile (~80% of the population; the other 20% exercise the empty state). Three consequences. A sim label is internally consistent arithmetic but **not real nutrition data** for the dish named on the card, so nothing may present one as a fact about food. Sim labels carry **no source**, so they read as manual and **recompute_auto_nutrition()** never touches them — correct, but it means the **backfill** has no sim coverage either; **supabase/tests/nutrition_fixtures.sql** is where it is exercised. The **hosted project now carries 29d's labels** (2026-08-25) but still has no sim — see the hosted rollout bullet below, which also corrects what this register used to claim about it.
- **Hosted was brought from Phase 27 to 29d on 2026-08-25 — and this register had it wrong.** It claimed hosted carried "the two all-10 placeholders and twelve nulls". It carried neither: Phase 28 had never been applied there, so **recipes.nutrition** did not exist as a column at all,

nor did `food`, `ingredients.food_id`, `estimate_nutrition` or `recompute_auto_nutrition`. The last apply had been 2026-08-23 (Phase 27). **A claim about a database you cannot see goes stale silently** — this one was two phases out of date and was being planned around. Measure first. What the rollout took, in order: `pg_dump` of the public schema as the only undo (no PITR on the free tier), `0001_init.sql`, `nutrition_foods.sql` (78 foods / 277 aliases / 174 portions), delete the 14 kitchen recipes, re-apply `seed_recipes.sql`, verify. The delete was safe because it was **measured, not assumed**: those 14 carried 0 likes, 0 saves, 0 shares, 0 forks and all sat at version 1, their visible like/save/view counters are authored by `seed_recipe_v2`, and their 41 rating rows are the seed file's own demo ratings — so everything came back. Total cost was 4 rows in `recipe_views` (3 distinct viewers, so `view_count` fell by 1 on three recipes). Verified after: 12 `auto` / 1 manual / 1 null, 137 linked ingredients, and **0 drift** between the committed labels and what the hosted registry recomputes. Note the general shape — because `seed_recipe_v2` is not an upsert (Gotcha 16), delete-and-reseed is the *only* way to push a content change to a database that already has the recipe, and whether that is acceptable is an engagement question to be answered with a query, not a guess.

- **The hosted project has no simulated population at all** — only `seed.sql` + `seed_recipes.sql`. Measured there 2026-08-23, straight after the schema apply: `recipes_quick` **10** rows, `recipes_projects` **1**, `recipes_most_forked` **0**. So `03 MOST FORKED` is legitimately empty on production until somebody forks something, and the shelf's own copy ("no public recipe has been forked") is the correct thing for a visitor to see there — it is not a bug report waiting to happen. Do **not** "fix" it by running `db:sim` against hosted: the sim writes ~1,000 `auth.users` rows and its teardown is registry-driven, so that is a one-way door on a real project. If hosted ever needs a populated fork tree, seed a handful of deliberate forks in `seed.sql` instead, where they are content rather than simulation.

BL-6 — environment-dependent verification gaps

Not code debt — things this machine cannot exercise. **Fork from the UI** and **Storage image upload** are still unexercised end-to-end; the **mobile/emulator** manual pass is blocked with no Android SDK installed. **Trigger**: a machine with the Android SDK, and a local-stack session for the two flows.

Screenshots are no longer on this list. Chrome was installed 2026-08-22, so the B028 procedure (release build + `npx serve` + Playwright) runs here; Phase 26 used it and Phase 23's outstanding pass was completed at the same time.

Dark mode has a procedure now (2026-08-23). The browser cannot emulate `prefers-color-scheme` after Flutter reads it at boot, so shoot the `app` instead of the browser: pin `themeMode: ThemeMode.dark` in `apps/app/lib/main.dart`, rebuild, screenshot, then revert and rebuild. Check `git diff` on `main.dart` is empty before you call it done — the pin is a two-line edit that is very easy to leave behind.

Re-shoot on a new port after a rebuild. `npx serve` plus the browser's HTTP cache will happily keep serving the *previous* build from the same origin: the light rebuild above rendered dark until it was served on a different port, with `matchMedia` reporting light the whole time. There is no service worker to blame (Flutter's is not registered in this build) — it is plain HTTP caching, and it will make a revert look like it did not take.

What still cannot be driven here is a **tap inside the Flutter canvas**: web semantics do not come up headlessly, so there are no DOM nodes to target and navigation has to be driven by URL.

BL-7 — the RLS acceptance matrix as a *signed-in* user — DONE (2026-08-23)

Was the single largest verification hole in the project. Everything that had ever exercised RLS here ran as `postgres`, which bypasses policies outright: `seed.sql`, the sim, `3_sim_verify.sql`, CI's `database.yml`, and every hosted check. `anon` was covered (Phase 26 proved it on hosted: the shelves are callable and a signed-out reader sees 22 of 24 recipes, the two private ones filtered). `authenticated` was not — exactly the gap **B053** lived in, where `recipes_select` called a `stable` function that re-queried its own table so *every recipe creation failed*, unnoticed for months, found by hand while doing something else.

- ☒ **The matrix is now a file:** `supabase/tests/rls_matrix.sql`, `melos run db:rls`. It creates three throwaway `auth.users` (owner / shared-with / unrelated stranger), a private and a public recipe with content, re-runs everything under `set local role authenticated + request.jwt.claims`, prints one PASS/FAIL line per check, and **rolls the transaction back** — no user, no recipe, no helper function survives it. **92 checks, all passing** on the local stack (79 + Phase 29a's \$E registry nine + 29b's B22b food link + 29c's B22c/B22d auto-estimate guards, the pair that found B075).
- ☒ **CI runs it** — a new `RLS matrix – as a signed-in user` step in `database.yml`, straight after the fresh apply. A B053/B061-class regression now fails a pull request.
- ☒ **It found a real hole on its first complete run: B061.** `likes_write` and `saves_write` checked only `user_id = auth.uid()`, with no read test — so any signed-in user who knew a private recipe's uuid could like or save it and move its `like_count` / `save_count`, which the owner then publishes when they flip it public. `ratings_write` and `views_insert` had `can_read_recipe` and these two did not. Fixed in `0001_init.sql`; reverting the policy turns the run red on exactly D18 and D20, which is how the check was proven non-vacuous.

What is covered, per role, against a **private** recipe, a **shared** private recipe, and a **public** one:

Role	select	insert (... RETURNING)	update	delete	RPCs
<code>anon</code>	☒	☒ must FAIL	—	—	☒ <code>fork_recipe</code> must FAIL
owner	☒	☒ (the B053 shape)	☒ + column grants must FAIL (B050)	☒	☒ <code>save_recipe</code> create/update; the five revoked helpers must FAIL
shared-with (<code>recipe_shares</code>)	☒ incl. content + versions	☒ must FAIL	☒ 0 rows, no error	☒ 0 rows, no error	☒ <code>save_recipe</code> must FAIL, <code>fork_recipe</code> allowed

Role	select	insert (... RETURNING)	update	delete	RPCs
unrelated signed-in user	☒ 0 rows	☒ must FAIL	☒ 0 rows, no error	☒ 0 rows, no error	☒ <code>fork_recipe</code> public allowed / private must FAIL

The two that fail *silently* and therefore mattered most are asserted as such: an `update/delete` matching zero rows returns success (Gotcha 2 — the server-side twin of B011), so those checks assert the **row count**, not the absence of an error; and a `select` policy that cannot see its own `INSERT ... RETURNING` row (B053) is written longhand so "raised" and "succeeded but returned nothing" are distinguishable. Both look like working code.

Trigger: any change to a policy, a `security definer` function, or the column grants — run `melos run db:rls`, and add a check to the file for any new table, policy, or definer function in the same change.

Still not covered by it: Storage RLS (the `recipe-images` / `avatars` bucket policies), which needs the storage container and an upload rather than SQL; and the PostgREST edge — the matrix talks to Postgres directly, so it proves the policies, not that postgrest-dart sends what the policies expect. `packages/core/test/`'s recording client is the other half of that pair.